МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
«УЖГОРОДСЬКИЙ УНІВЕРСИТЕТ»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ СИСТЕМ

Лабораторна робота № 1

Апроксимація функцій методами інтерполяції та МНК

Виконав:
студент 4-го курсу
денної форми навчання
групи ІПЗ 2
Подзивалов Руслан Олександрович

Ужгород – 2026

Мета роботи: засвоїти підходи до апроксимації даних — інтерполяційний поліном Лагранжа та метод найменших квадратів (МНК), набути практичних навичок їх реалізації мовою JavaScript і візуалізації результатів, а також пройти повний цикл розробки програмного продукту: від технічного завдання та вибору методології до тестування, документування і звіту про використання штучного інтелекту.

План лабораторної роботи:

– 1. Розробка технічного завдання (ТЗ).

– 2. Вибір методології розробки.

– 3. Проєктування логіки: блок-схеми та таблиці ідентифікаторів.

– 4. Програмна реалізація.

– 5. Аналітичне тестування для різної кількості даних.

– 6. Моделювання взаємодії компонентів.

– 7. Документування та репозиторій GitHub.

– 8. Звіт про використання AI (AI Interaction Log).

Лабораторне завдання:

1) **Варіант:** точки (0; 0,0), (1; 0,5), (2; 0,8), (3; 0,5), (4; 0,0).

2) **Програмний продукт:** забезпечити вибір і введення набору $\{(x_i, y_i)\}$, побудову інтерполяційного полінома Лагранжа (або Ньютона з таблицею поділених різниць), МНК-апроксимацію поліномом степеня m через стійкий метод лінійної алгебри (без нормальних рівнянь у підсумкових розрахунках), обчислення залишків $r_i = y_i - \hat{y}_i$, графіки точок, кривих і діаграму залишків, перемикач «Інтерполяція / МНК / Усі».

3) **Анімація:** покрокове додавання вузлів інтерполяції з плавним оновленням кривої та виділенням активного вузла; «вирощування» МНК-полінома від горизонтальної лінії й поступова поява залишків.

4) **Примітки викладача:** обчислення для 5, 10 і 20 точок; 6 графіків (3 — Лагранж, 3 — МНК); автоматичне масштабування осей X і Y за обчисленими min і max, щоб графік завжди лишався у видимій зоні; миттєве перемикання між наборами 5/10/20.

ХІД РОБОТИ

1. Розробка технічного завдання

Призначення. Веб-застосунок за табличними даними будує інтерполяційний поліном Лагранжа (з таблицею поділених різниць у формі Ньютона) і МНК-апроксимацію поліномом степеня m, анімує побудову, показує залишки й метрики та дає змогу миттєво порівнювати результати для 5, 10 і 20 точок. Застосунок працює повністю в браузері і відкривається подвійним клацанням по index.html, без сервера та зовнішніх бібліотек.

Вхідні дані. У файлі з методичними вказівками є лише набір із 5 точок варіанта, а додатка з наборами на 10 і 20 точок у ньому немає. Тому ці набори згенеровано програмно (scripts/gen-datasets.js) на тому ж відрізку [0; 4] за формою варіанта. За «базову форму» взято інтерполяційний поліном самого варіанта $g(x) = 0{,}8 − (x − 2)^2/3 + (x − 2)^4/30$, вузли рівновіддалені, до значень додано детермінований «шум вимірювань» $\varepsilon \in [-0{,}03;\ 0{,}03]$ (фіксований seed), результат округлено до 0,001. Будь-який набір можна замінити через форму введення в інтерфейсі.

Таблиця 1 – Набори даних для 5, 10 і 20 точок

i	x (5)	y (5)	x (10)	y (10)	x (20)	y (20)
0	0	0,0	0	−0,003	0	−0,012
1	1	0,5	0,444	0,177	0,211	0,09
2	2	0,8	0,889	0,449	0,421	0,193
3	3	0,5	1,333	0,665	0,632	0,323
4	4	0,0	1,778	0,763	0,842	0,425
5			2,222	0,767	1,053	0,556
6			2,667	0,669	1,263	0,623
7			3,111	0,457	1,474	0,714
8			3,556	0,176	1,684	0,777
9			4	−0,027	1,895	0,824
10					2,105	0,797
11					2,316	0,738
12					2,526	0,733
13					2,737	0,621
14					2,947	0,527
15					3,158	0,426
16					3,368	0,296
17					3,579	0,19

i	x (5)	y (5)	x (10)	y (10)	x (20)	y (20)
18					3,789	0,085
19					4	0,007

Таблиця 2 – Функціональні вимоги та спосіб їх перевірки

ID	Вимога	Перевірка
ФВ-01	Завантаження/вибір наборів 5, 10, 20 точок і введення власного набору у формі	UI-02, UI-05, T-15
ФВ-02	Інтерполяційний поліном Лагранжа (формула 4.3)	T-01…T-04
ФВ-03	Таблиця поділених різниць і поліном Ньютона (формула 4.4)	T-05, T-06
ФВ-04	МНК: дизайн-матриця степеня m (1…6), розв'язок QR-розкладом Хаусхолдера, без нормальних рівнянь	T-07…T-12
ФВ-05	Розрахункові значення $\hat{y}_i$, залишки $r_i = y_i - \hat{y}_i$, метрики SSE, RMSE, R^2, max|r|, $cond_2(A)$	T-09, T-13, T-16
ФВ-06	Графік: точки, крива інтерполяції, крива МНК; перемикач «Інтерполяція / МНК / Усі»	UI-02
ФВ-07	Діаграма залишків (штрих-графік, маркери, підписана нульова лінія)	огляд, скріншоти
ФВ-08	Анімація Лагранжа: покрокове додавання вузлів, плавне оновлення кривої, виділення активного вузла	UI-03, скріншоти
ФВ-09	Анімація МНК: «вирощування» від горизонталі $\bar{y}$ до кривої, поступова поява залишків	скріншоти
ФВ-10	Автоматичне масштабування осей X і Y за min/max усіх видимих об'єктів у кожному кадрі	M-01…M-06, UI-03
ФВ-11	Миттєве перемикання 5/10/20 (кнопки та клавіші 1/2/3)	UI-02
ФВ-12	6 графіків: 3 для Лагранжа і 3 для МНК (5/10/20) + порівняльна таблиця	UI-06
ФВ-13	Підказки при наведенні: координати, значення поліномів, похибка/залишок у точці	скріншоти
ФВ-14	Демонстрація екстраполяції (зона ±25 % за межами вузлів)	скріншот

Вимоги до інтерфейсу: адаптивне полотно (ResizeObserver, урахування devicePixelRatio), робота від ширини 360 px без горизонтальної прокрутки; осі з підписами й «гарними» поділками $1\cdot10^k$, $2\cdot10^k$, $5\cdot10^k$, сітка, легенда; контрастні кольори (Лагранж — синій, МНК — помаранчевий, залишки — фіолетовий, активний вузол — червоний); світла і темна теми; усі елементи керування — в одному рядку над графіком; клавіші 1/2/3 для перемикання наборів і пробіл для анімації.

Обмеження системи: вузли x_i попарно різні, щонайменше 2 точки; степінь МНК $m \le n - 1$; глобальний поліном високого степеня на рівновіддалених вузлах осцилює біля країв (ефект Рунге) — це властивість

методу, яку програма має чесно показувати; чисельна стійкість МНК забезпечується нормалізацією $x \to t \in [-1; 1]$ і QR-розкладом.

Нефункціональні вимоги: перерахунок моделі й кривих менше ніж за 1 мс для $n \leq 20$; плавна анімація (requestAnimationFrame, 60 кадр/с); автоматизовані тести (npm test) та аналітичне тестування (npm run bench).

2. Вибір методології

Для розробки обрано **V-модель** — розширення каскадної моделі, у якій кожному етапу специфікації відповідає свій рівень перевірки. Причини вибору:

– вимоги стабільні й повністю задані методичкою та варіантом, тож ітеративне уточнення із замовником, заради якого обирають Agile, тут не потрібне;

– результат математично перевірюваний, тому критерії перевірки для кожного рівня можна сформулювати заздалегідь (проходження через вузли, ортогональність залишків, межі осей тощо);

– обсяг невеликий, ризики низькі, тож спіральна модель з окремим аналізом ризиків на кожному витку була б надлишковою.

Таблиця 3 – Відповідність фаз V-моделі та рівнів тестування

Фаза специфікації	**Артефакт**	**Рівень перевірки**	**Артефакт перевірки**
V1. Вимоги (ТЗ)	docs/TZ.md	V7. Приймальне тестування	docs/analysis.md, скріншоти, таблиця 2
V2. Архітектура	діаграми компонентів і взаємодії	V6. Інтеграційне тестування	tests/ui.test.js (UI-01… UI-06)
V3. Детальне проєктування	блок-схеми, таблиці ідентифікаторів	V5. Модульне тестування	tests/approx.test.js, tests/scale.test.js
V4. Кодування	js/*.js, index.html, css/style.css	—	—

Кожну фазу зафіксовано окремим комітом із префіксом фази ([V1 Вимоги], [V4 Кодування], [V5 Модульні тести] тощо); повна історія комітів наведена в розділі 7.

3. Проєктування логіки

3.1. Математична модель

Інтерполяція Лагранжа: $p_n(x) = \Sigma_k\, y_k \cdot L_k(x)$, $L_k(x) = \Pi_{j \neq k}\, (x - x_j)/(x_k - x_j)$ (формула 4.3). Для перевірки та виводу формули додатково будується таблиця поділених різниць і поліном Ньютона $p_n(x) = a_0 + a_1(x - x_0) + \ldots + a_n\Pi(x - x_j)$, $a_k = f[x_0, \ldots, x_k]$ (формула 4.4).

Для 5 точок варіанта поділені різниці першого порядку дорівнюють 0,5; 0,3; −0,3; −0,5, другого — −0,1; −0,3; −0,1, третього — −1/15; 1/15, четвертого — 1/30. Звідси $p_4(x) = 0{,}8 - (x - 2)^2/3 + (x - 2)^4/30 = 0{,}0333x^4 - 0{,}2667x^3 + 0{,}4667x^2 + 0{,}2667x$: поліном симетричний відносно x = 2, як і самі дані.

МНК: шукаємо $\theta = \operatorname{argmin} \Sigma(y_i - g(x_i; \theta))^2$, g — поліном степеня m. Щоб зменшити обумовленість, змінну нормалізовано: $t = (x - c)/h$, $c = (x_0 + x_n)/2$, $h = (x_n - x_0)/2$, тож $t \in [-1; 1]$. Дизайн-матриця $A[i][j] = t_i^j$ (j = 0…m) розкладається відбиттями Хаусхолдера A = QR; θ знаходиться зворотною підстановкою з $R\theta = (Q^T y)_{0\ldots m}$. Нормальні рівняння $A^TA\theta = A^Ty$ не використовуються, бо $\text{cond}(A^TA) = \text{cond}(A)^2$, і на високих степенях це руйнує точність (див. розділ 5, таблиця 12). Після розв'язання θ перераховується в коефіцієнти за степенями x.

Для варіанта при m = 2 дані симетричні, тому лінійний член у t зникає: $g(x) = a + b(x - 2)^2$, $b = \Sigma(s - \bar{s})(y - \bar{y})/\Sigma(s - \bar{s})^2 = -2{,}6/14 = -13/70$, $a = \bar{y} - 2b = 0{,}36 + 26/70$, тобто $g(x) \approx -0{,}0114 + 0{,}7429x - 0{,}1857x^2$. Програма дає ті самі коефіцієнти (тест Т-08). З тієї ж причини m = 3 дає такий самий результат, як m = 2, тому за замовчуванням у програмі обрано m = 2.

Автомасштабування осей: у кожному кадрі програма обчислює min і max по всіх видимих об'єктах: експериментальних точках і кожній видимій кривій (300 відліків на її поточному відрізку, нескінченні значення відкидаються). До розмаху додається відступ 3 % по X і 12 % по Y. Поточні межі плавно наближаються до цільових: bounds ← bounds + (target − bounds)·(1 − e^(−10·dt)). Тому під час анімації і перемикання наборів осі

«під'їжджають» до нових меж, а не стрибають, і крива завжди лишається у видимій зоні.

3.2. Блок-схеми алгоритмів

Рисунок 1 – Блок-схема загального алгоритму роботи програми

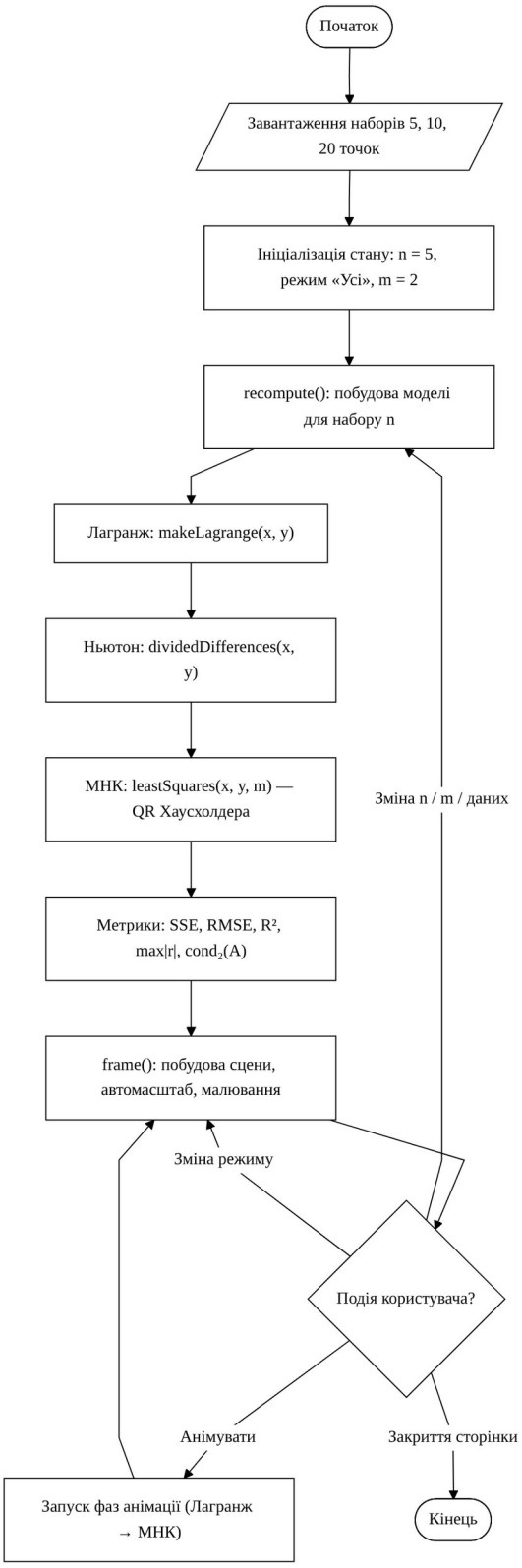


Рисунок 2 – Блок-схема обчислення полінома Лагранжа lagrange(xs, ys, x)

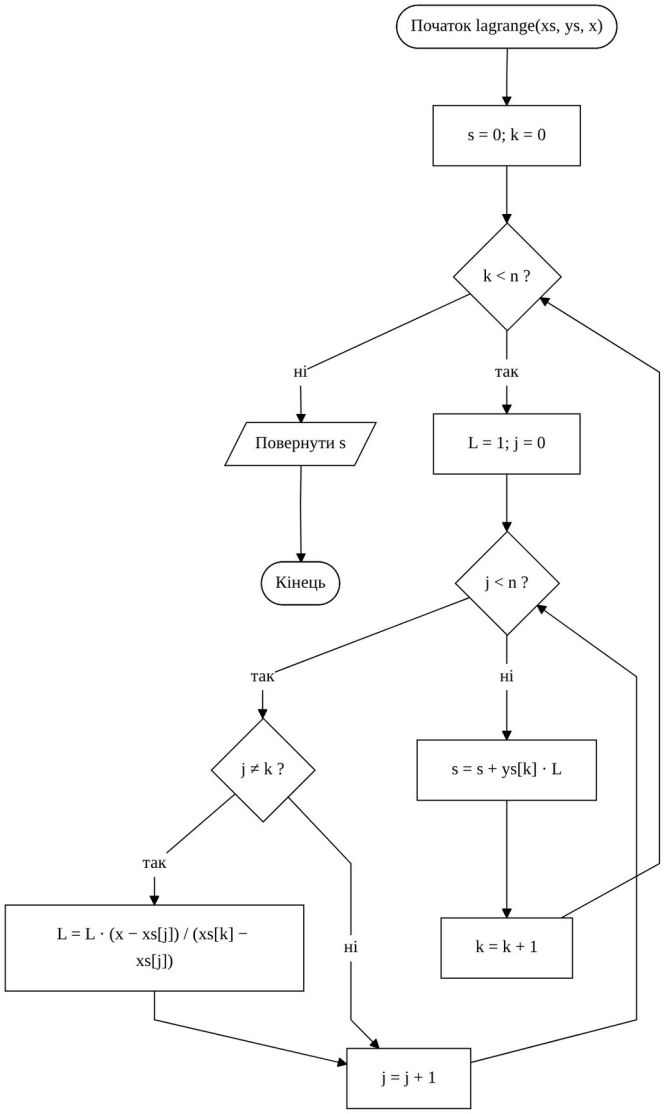


Рисунок 3 – Блок-схема МНК через QR-розклад Хаусхолдера leastSquares(xs, ys, m)

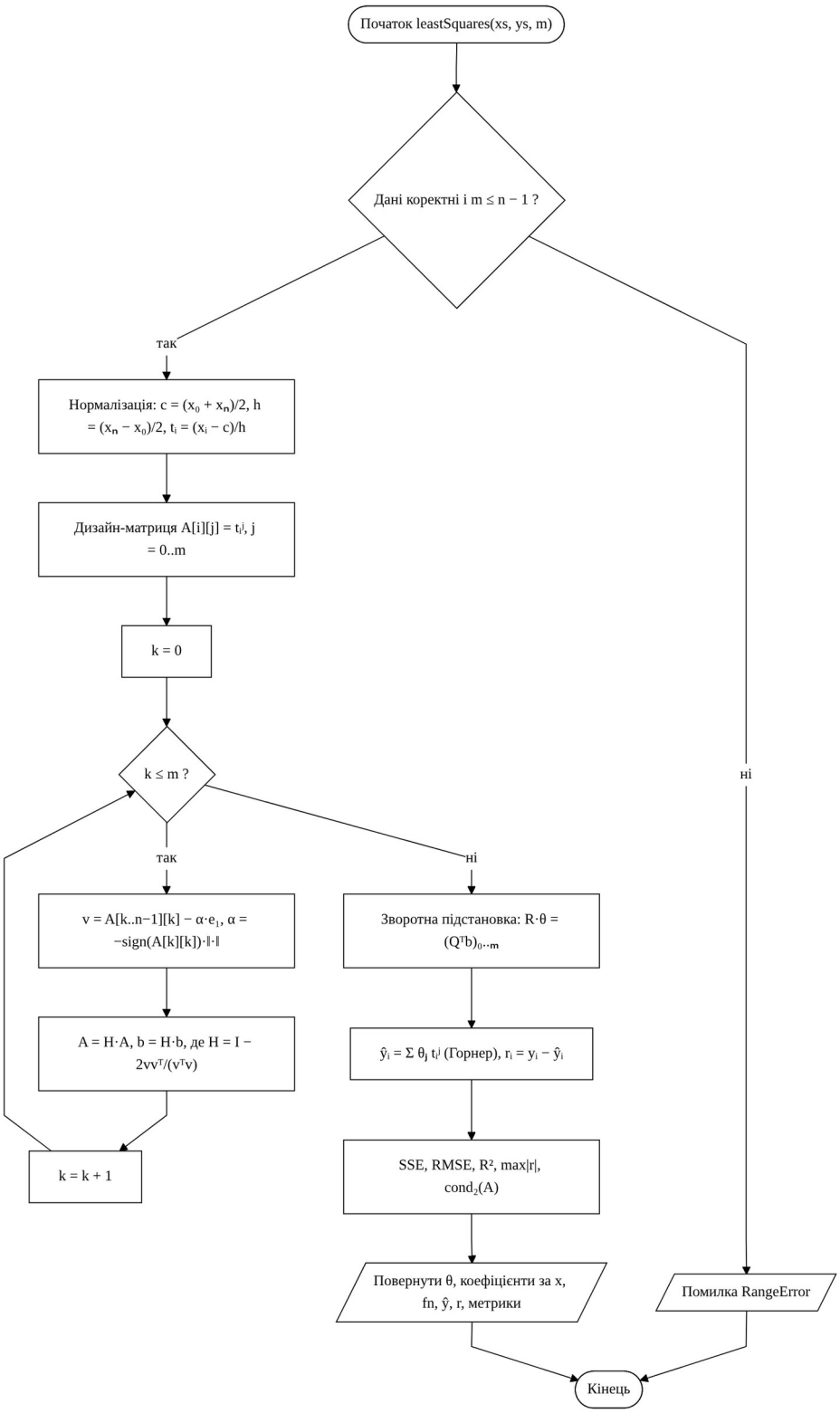
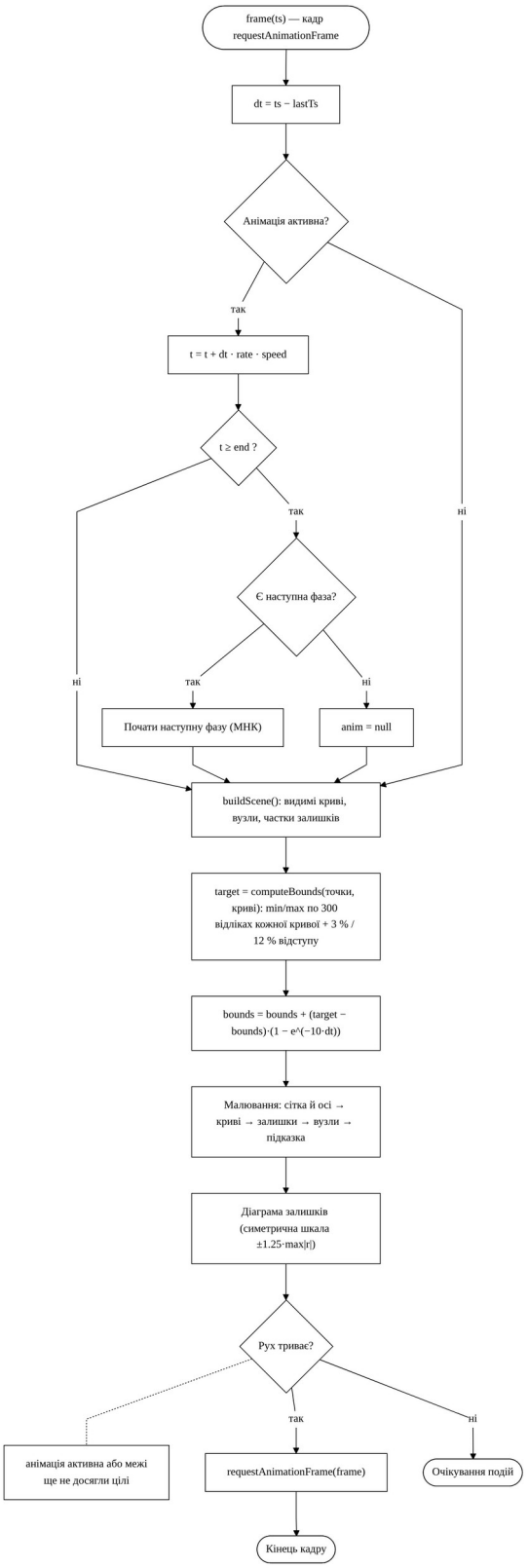


Рисунок 4 – Блок-схема кадру анімації з автоматичним масштабуванням осей frame(ts)



3.3. Таблиці ідентифікаторів

Таблиця 4 – Ідентифікатори змінних і констант

Ідентифікатор	Тип	Модуль	Призначення
xs, ys	number[]	approx.js	вузли x_i та значення y_i табличної функції
n	number	усі	кількість точок у наборі
k, j	number	approx.js	індекси вузлів у формулі Лагранжа (L_k, добуток за $j \neq k$)
m, mEff	number	approx.js, app.js	заданий і фактичний ($\leq n-1$) степінь МНК-полінома
c, h	number	approx.js	центр і напівширина відрізка для нормалізації $t = (x - c)/h$
ts	number[]	approx.js	нормалізовані вузли $t_i \in [-1; 1]$
A	number[][]	approx.js	дизайн-матриця (Вандермонда) розміру $n \times (m + 1)$
R, qtb	number[][], number[]	approx.js	верхньотрикутна матриця R та вектор Q^Tb після відбиттів Хаусхолдера
v, alpha	number[], number	approx.js	вектор і коефіцієнт відбиття $H = I - 2vv^T/(v^Tv)$
theta	number[]	approx.js	коефіцієнти МНК-полінома за степенями t
coeffsX	number[]	approx.js	коефіцієнти МНК-полінома за степенями x
fitted, residuals	number[]	approx.js	розрахункові значення $\hat{y}_i$ і залишки $r_i = y_i - \hat{y}_i$
sse, rmse, r2, maxAbs	number	approx.js	метрики: Σr^2, $\sqrt{(SSE/n)}$, коефіцієнт детермінації, $\max\|r_i\|$
cond	number	approx.js	число обумовленості $cond_2(A) = \sigma max/\sigma min$
table, coeffs	number[][], number[]	approx.js	таблиця поділених різниць і коефіцієнти Ньютона a_k
state	object	app.js	стан застосунку: n, mode, m, normalize, extrap, speed, sets, model, anim, bounds, hover
state.anim	object | null	app.js	поточна фаза анімації: kind, t, end, rate, queue, paused
bounds, target	object	app.js, scale.js	поточні та цільові межі осей {xMin, xMax, yMin, yMax}
padX, padY	number	scale.js	відносні відступи меж (0.03 і 0.12 від розмаху)
s, g	number	app.js	параметр плавного переходу (smoothstep) у кадрі анімації
residFrac	number[]	app.js	частка «намальованості» кожного залишку (0…1)
ORIGINAL	object	app.js	незмінні вихідні набори 5/10/20 (для кнопки «Повернути»)

Таблиця 5 – Ідентифікатори функцій

Функція	Модуль	Вхід	Результат
validatePoints(xs, ys)	approx.js	масиви точок	виняток, якщо дані некоректні
lagrangeBasis(xs, k, x)	approx.js	вузли, індекс, точка	$L_k(x)$
lagrange(xs, ys, x)	approx.js	вузли, значення, точка	$p_n(x)$ за формулою 4.3
makeLagrange(xs, ys)	approx.js	набір	функція $x \mapsto p_n(x)$

Функція	Модуль	Вхід	Результат
dividedDifferences(xs, ys)	approx.js	набір	{table, coeffs} — поділені різниці
newtonEval(xs, coeffs, x)	approx.js	вузли, a_k, точка	значення полінома Ньютона (Горнер)
normalization(xs, enabled)	approx.js	вузли	{c, h}
designMatrix(ts, m)	approx.js	t_i, степінь	матриця A
householderLeastSquares(A, b)	approx.js	A, b	θ — МНК-розв'язок через QR
leastSquares(xs, ys, m, opt)	approx.js	набір, m	модель МНК: θ, coeffsX, fn, fitted, residuals, metrics, cond
normalEquationsSolve(...)	approx.js	набір, m	розв'язок через A^TA (лише для порівняння стійкості)
conditionNumber(A)	approx.js	матриця	$cond_2(A)$ (обертання Якобі для A^TA)
metrics(ys, fitted)	approx.js	y, ŷ	{sse, rmse, r2, maxAbs, mean}
maxDeviation(f, g, a, b)	approx.js	дві функції, відрізок	max|f − g| на сітці
rangeOf(f, a, b)	approx.js	функція, відрізок	{min, max}
parsePoints(text)	approx.js	текст форми	{x, y}, відсортовані за x
computeBounds(points, curves, opt)	scale.js	видимі об'єкти	межі осей з відступами
niceTicks(lo, hi, target)	scale.js	діапазон	{ticks, step}
approach(cur, target, k)	scale.js	межі	плавно наближені межі
buildModel(set, m, normalize)	app.js	набір	усі обчислення для набору + час
buildScene()	app.js	стан	список кривих, точок і залишків поточного кадру
frame(ts)	app.js	час кадру	крок анімації, автомасштаб, малювання
play(), pause(), stopAnimation()	app.js	—	керування анімацією
setDataset(n), setMode(mode)	app.js	5/10/20; режим	перемикання набору / режиму
renderInfo(), renderComparison()	app.js	—	таблиці, формули, 6 мініграфіків
onMove(ev), onLeave()	app.js	подія миші	підказка і приціл

Таблиця 6 – Ідентифікатори об'єктів (класів і модулів)

Об'єкт	Тип	Відповідальність
Approx	модуль (UMD)	чисельні методи: Лагранж, Ньютон, МНК, метрики, розбір введення
Scale	модуль (UMD)	автомасштаб і поділки осей
Plot	клас	рендерер Canvas 2D: осі, сітка, криві, точки, залишки, приціл
DATASETS	модуль (UMD)	набори 5/10/20 і еталонна форма g(x)
app (IIFE)	модуль	стан, події UI, анімація, зв'язок модулів
mainPlot, resPlot, miniPlots	екземпляри Plot	основний графік, діаграма залишків, 6 мініграфіків

4. Програмна реалізація

Стек: JavaScript (ES2020), HTML5 Canvas 2D, CSS (custom properties для світлої й темної тем); Node.js 22 з вбудованим node:test для модульних тестів; Playwright для браузерних тестів і скріншотів; Mermaid для діаграм; Git для контролю версій. Зовнішніх бібліотек у самому застосунку немає. Модулі написано у форматі UMD (класичні скрипти, а не ES-модулі), тому сторінка працює і при відкритті файлом (file://), і ті самі файли підключаються в Node.js для тестів.

Таблиця 7 – Структура програмного продукту

Файл	Рядків	Призначення
js/approx.js	404	математичне ядро: Лагранж, Ньютон, МНК (QR Хаусхолдера), $cond_2$, метрики, розбір введення
js/scale.js	104	автомасштаб осей, «гарні» поділки, плавне наближення меж
js/chart.js	305	клас Plot: Canvas, сітка, осі, криві, точки, залишки, приціл, діаграма залишків
js/app.js	546	контролер: стан, анімація (requestAnimationFrame), підказки, панелі, 6 мініграфіків
js/datasets.js	38	набори 5/10/20 точок і базова форма g(x)
index.html, css/style.css	—	розмітка інтерфейсу, адаптивна верстка, теми
tests/*.test.js	—	30 автоматизованих тестів (модульні + UI)
scripts/*.js	—	генератор наборів, бенчмарк (аналітичне тестування), знімання скріншотів

Анімація Лагранжа. Параметр t зростає від 0 до n − 1 (≈0,7 с на вузол). На кроці k уже додано k + 1 вузлів; крива є плавною сумішшю двох проміжних поліномів $(1 - s)\cdot P_k(x) + s\cdot P_{k+1}(x)$, $s = \text{smoothstep}(t - k)$, і малюється на відрізку $[x_0;\ x_k + s(x_{k+1} - x_k)]$, тобто «тягнеться» до наступного вузла. Активний вузол збільшено й пофарбовано в червоний, ще не додані вузли показано порожніми кружками.

Анімація МНК. На першій фазі крива плавно переходить від горизонталі $\bar{y}$ до МНК-полінома: $(1 - g)\cdot\bar{y} + g\cdot g(x)$; на другій — по черзі «виростають» відрізки залишків від $(x_i,\ \hat{y}_i)$ до $(x_i,\ y_i)$ і стовпці діаграми залишків. У режимі «Усі» фази йдуть одна за одною.

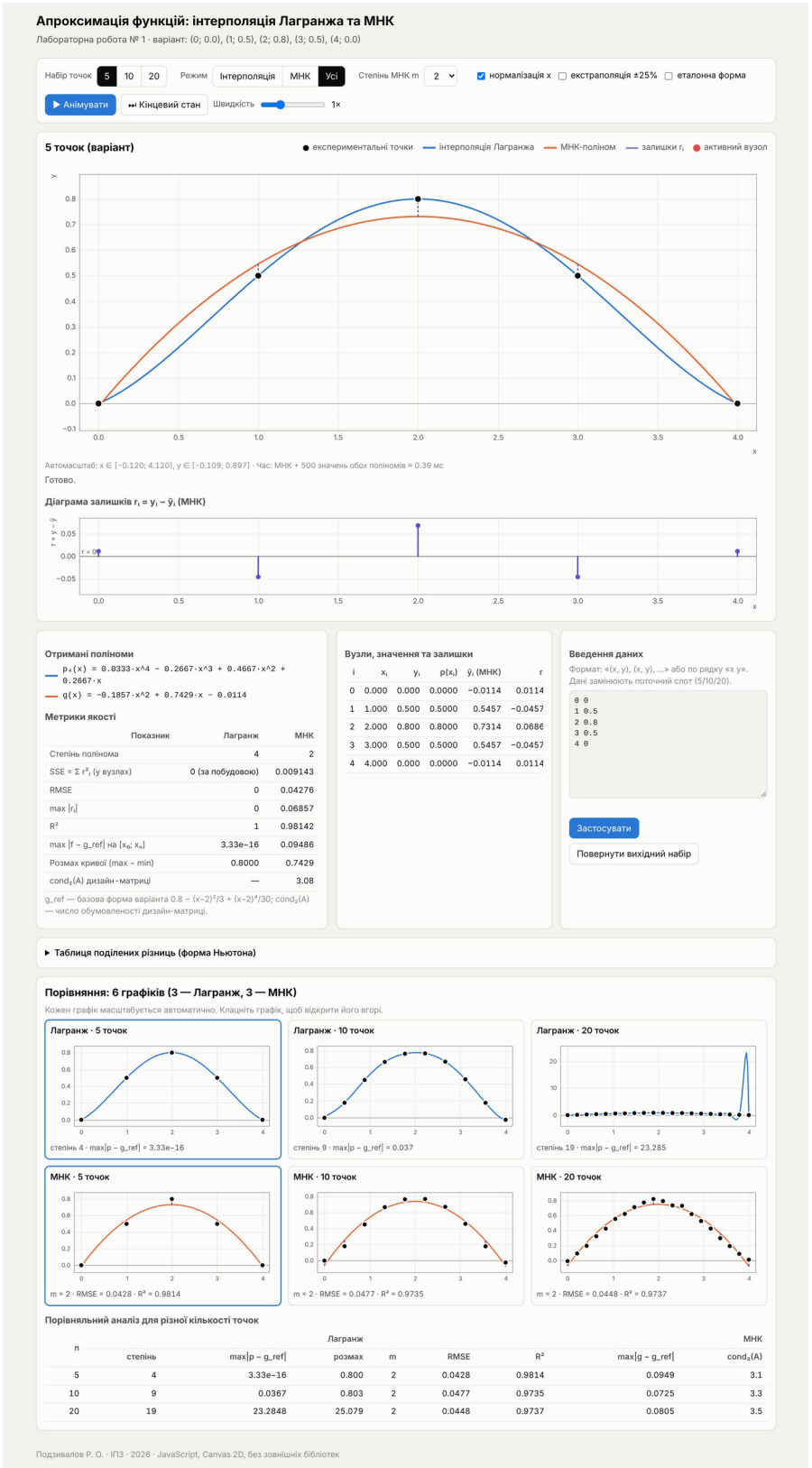
Ключовий фрагмент — розв'язання МНК відбиттями Хаусхолдера (повний код — у додатку Б):

```
for (let k = 0; k < cols; k++) {
  let norm = 0;                          // норма k-го стовпця нижче діагоналі
  for (let i = k; i < rows; i++) norm += R[i][k] * R[i][k];
  norm = Math.sqrt(norm);
  const alpha = R[k][k] > 0 ? -norm : norm;  // знак проти скорочення
  const v = new Array(rows).fill(0);
  for (let i = k; i < rows; i++) v[i] = R[i][k];
  v[k] -= alpha;                          // v = x − alpha·e1
  let vv = 0; for (let i = k; i < rows; i++) vv += v[i] * v[i];
  for (let j = k; j < cols; j++) {     // R ← H·R
    let s = 0; for (let i = k; i < rows; i++) s += v[i] * R[i][j];
    const f = (2 * s) / vv; for (let i = k; i < rows; i++) R[i][j] -= f *
v[i];
  }
  /* так само b ← H·b */
}
// зворотна підстановка R·θ = (Qᵀb)[0..m]
```

Ключовий фрагмент автомасштабування (кожен кадр):

```
const target = Scale.computeBounds([{ xs: M.x, ys: M.y }], scene.curves,
                                   { padX: 0.03, padY: 0.12 });
state.bounds = Scale.approach(state.bounds, target, 1 - Math.exp(-dt * 10));
if (!Scale.boundsClose(state.bounds, target)) busy = true;  // ще один кадр
mainPlot.bounds = state.bounds;
```

Рисунок 5 – Загальний вигляд застосунку (5 точок, режим «Усі», m = 2)



На рисунку видно панель керування (набір 5/10/20, режим, степінь m, нормалізація, екстраполяція, анімація), основний графік з легендою, рядок автомасштабу з поточними межами осей, діаграму залишків, панелі з

поліномами, метриками, таблицею вузлів і формою введення, а внизу — 6 графіків порівняння та порівняльну таблицю.

5. Аналітичне тестування

5.1. Автоматизовані тести

Тести запускаються командою npm test; усі 30 тестів проходять (Node.js 22, Playwright 1.63). Модульні тести (рівень V5) перевіряють ядро та автомасштаб на ручних розрахунках і математичних властивостях, UI-тести (рівень V6) — роботу інтерфейсу в headless-браузері.

Таблиця 8 – Перелік автоматизованих тестів

ID	Що перевіряється	Рівень	Результат
T-01	Лагранж проходить через усі вузли варіанта (5, 10, 20 точок)	V5, модуль	пройдено
T-02	Базисні поліноми: L_k(x_j) = δ_kj і Σ L_k(x) = 1	V5, модуль	пройдено
T-03	Поліном варіанта: p(x) = 0.8 − (x−2)²/3 + (x−2)⁴/30 (ручний розрахунок)	V5, модуль	пройдено
T-04	Лагранж точно відтворює поліноми степеня ≤ n−1	V5, модуль	пройдено
T-05	Ньютон збігається з Лагранжем (поділені різниці)	V5, модуль	пройдено
T-06	Таблиця поділених різниць для варіанта (ручний розрахунок)	V5, модуль	пройдено
T-07	МНК: пряма через точні дані відновлюється точно	V5, модуль	пройдено
T-08	МНК для варіанта, m = 2 (ручний розрахунок нормальних рівнянь)	V5, модуль	пройдено
T-09	МНК: сума залишків = 0 і залишки ортогональні до стовпців матриці	V5, модуль	пройдено
T-10	МНК при m = n−1 збігається з інтерполяцією Лагранжа	V5, модуль	пройдено
T-11	QR (Хаусхолдер) збігається з нормальними рівняннями на добре обумовленій задачі	V5, модуль	пройдено
T-12	Нормалізація x: результат не залежить від неї, а обумовленість R значно краща	V5, модуль	пройдено
T-13	Метрики: $R^2 \in [0;1]$, RMSE = √(SSE/n), МНК m=4 краще за m=2	V5, модуль	пройдено
T-14	Валідація: однакові вузли, різна довжина, нечислові значення, завеликий степінь	V5, модуль	пройдено
T-15	Розбір введення: формат варіанта, рядки, десяткова кома, сортування	V5, модуль	пройдено
T-16	Число обумовленості: одинична матриця → 1, діагональ diag(1, 10) → 10	V5, модуль	пройдено
T-17	Перерахунок коефіцієнтів t → x і форматування полінома	V5, модуль	пройдено
M-01	computeBounds охоплює всі точки та всі значення кривої з відступом	V5, модуль	пройдено
M-02	Межі змінюються при перемиканні наборів 5 → 20 (осциляція Рунге видима)	V5, модуль	пройдено
M-03	Вироджені діапазони: одна точка та горизонтальна лінія не дають нульової висоти	V5, модуль	пройдено
M-04	NaN/Infinity у кривій ігноруються	V5, модуль	пройдено
M-05	niceTicks: крок 1·10^k / 2·10^k / 5·10^k, поділки в межах діапазону	V5, модуль	пройдено

ID	Що перевіряється	Рівень	Результат
M-06	approach: плавне наближення збігається до цілі	V5, модуль	пройдено
UI-01	стартовий стан: 5 точок, режим «Усі», без помилок	V6, UI	пройдено
UI-02	перемикання 5/10/20 миттєво змінює модель і межі осей	V6, UI	пройдено
UI-03	під час анімації межі завжди охоплюють видиму криву	V6, UI	пройдено
UI-04	клацання поза кнопками не зупиняє анімацію (регресія)	V6, UI	пройдено
UI-05	введення власного набору та повідомлення про помилку	V6, UI	пройдено
UI-06	6 мініграфіків намальовано, на мобільній ширині немає горизонтальної прокрутки	V6, UI	пройдено

5.2. Порівняльний аналіз для 5, 10 і 20 точок

Таблиця 9 – Результати для різної кількості точок (g_ref — базова форма варіанта)

Показник	n = 5	n = 10	n = 20
Степінь полінома Лагранжа	4	9	19
$\max\|p - g_ref\|$ на [0; 4]	$3{,}33 \cdot 10^{-16}$	0,0367	23,2880
Діапазон значень p(x) на [0; 4]	[0; 0,800]	[−0,027; 0,776]	[−1,779; 23,304]
LOOCV RMSE Лагранжа (прогноз виключеної точки)	0,1805	0,0255	19,3177
МНК m = 2: SSE	0,00914	0,02277	0,04010
МНК m = 2: RMSE	0,0428	0,0477	0,0448
МНК m = 2: R^2	0,9814	0,9735	0,9737
МНК m = 2: $\max\|r_i\|$	0,0686	0,0765	0,0875
МНК m = 2: $\max\|g - g_ref\|$ на [0; 4]	0,0949	0,0725	0,0805
LOOCV RMSE МНК	0,0972	0,0619	0,0451
$\text{cond}_2(A)$ (з нормалізацією)	3,08	3,31	3,50
max|Ньютон − Лагранж|	$3{,}61 \cdot 10^{-16}$	$2{,}99 \cdot 10^{-15}$	$1{,}49 \cdot 10^{-10}$

Висновки з таблиці. Для 5 точок інтерполяційний поліном збігається з базовою формою (похибка $3{,}33 \cdot 10^{-16}$ — машинний нуль), бо саме він і є базовою формою. Для 10 точок поліном 9-го степеня ще поводиться стабільно ($\max|p - g_ref| = 0{,}0367$), але вже «ловить» шум. Для 20 точок поліном 19-го степеня проходить через усі вузли, проте між крайніми вузлами різко осцилює: максимум сягає 23,30 при даних у межах [−0,012; 0,824] (ефект Рунге, підсилений шумом). Прогноз виключеної точки (LOOCV) показує те саме: 19,32 для Лагранжа проти 0,0451 для МНК. МНК із m = 2 дає стабільний результат для всіх n: RMSE ≈ 0,043–0,048, R^2 ≈ 0,97–0,98, похибка відносно базової форми не перевищує 0,095, і зі зростанням n МНК-оцінка не погіршується, а прогнозна похибка зменшується.

Таблиця 10 – Вплив степеня МНК-полінома m

n	m	SSE	RMSE	R^2	max|g − g_ref|
5	1	0,492000	0,3137	1,11e−16	0,4400
5	2	0,009143	0,0428	0,9814	0,0949
5	3	0,009143	0,0428	0,9814	0,0949
5	4	1,48e−31	1,72e−16	1,0000	4,44e−16
10	1	0,858388	0,2930	9,84e−5	0,4139
10	2	0,022771	0,0477	0,9735	0,0725
10	3	0,022527	0,0475	0,9738	0,0796
10	4	0,001131	0,0106	0,9987	0,0304
10	5	0,001127	0,0106	0,9987	0,0308
10	6	4,19e−5	0,0020	1,0000	0,0353
20	1	1,523196	0,2760	4,62e−5	0,4497
20	2	0,040096	0,0448	0,9737	0,0805
20	3	0,039582	0,0445	0,9740	0,0842
20	4	0,002954	0,0122	0,9981	0,0214
20	5	0,002925	0,0121	0,9981	0,0223
20	6	0,002793	0,0118	0,9982	0,0229

Через симетрію даних m = 3 практично не покращує m = 2, а m = 4 різко зменшує похибку (для 20 точок RMSE падає з 0,045 до 0,012, а відхилення від базової форми — з 0,080 до 0,021), бо базова форма є поліномом 4-го степеня. Подальше збільшення m (5, 6) лише підганяє шум: SSE майже не зменшується, а відхилення від базової форми зростає. При m = n − 1 МНК вироджується в інтерполяцію (n = 5, m = 4: SSE = 0). У програмі степінь вибирається зі списку, тож це можна побачити наживо; систематична «W»-подібна структура залишків при m = 2 (рисунок 11) — ознака того, що степінь замалий.

Таблиця 11 – Час обчислень (Node.js v22.23.4)

n	Лагранж: 1000 значень, мкс	Ньютон: таблиця + 1000 значень, мкс	МНК: побудова, мкс	МНК: 1000 значень, мкс
5	42,2	7,6	3,88	3,8
10	129,8	10,7	3,33	4,8
20	404,8	33,9	5,07	4,7

Обчислення за формулою Лагранжа має складність $O(n^2)$ на одне значення, тому час для 1000 значень зростає майже вдесятеро від n = 5 до n = 20. Форма Ньютона після одноразової побудови таблиці ($O(n^2)$) обчислюється схемою Горнера за O(n) і для 20 точок у 12 разів швидша. МНК-поліном степеня 2 обчислюється за O(m) незалежно від n. Усі

обчислення займають частки мілісекунди, тож перемикання наборів і анімація відбуваються без затримок.

Таблиця 12 – Чисельна стійкість (20 точок, еталон — QR з нормалізацією)

m	**$cond_2(A)$ з нормалізацією**	**$cond_2(A)$ без нормалізації**	**Похибка QR без нормалізації**	**Похибка нормальних рівнянь без нормалізації**
3	7,5	242,8	$1{,}18\cdot10^{-15}$	$2{,}14\cdot10^{-14}$
6	91,9	$1{,}66\cdot10^{5}$	$6{,}54\cdot10^{-15}$	$1{,}25\cdot10^{-11}$
9	1318,2	$1{,}69\cdot10^{8}$	$5{,}91\cdot10^{-13}$	$3{,}57\cdot10^{-7}$
12	$2{,}45\cdot10^{4}$	$> 10^{16}$	$5{,}65\cdot10^{-11}$	0,0042
15	$6{,}68\cdot10^{5}$	$> 10^{16}$	$7{,}91\cdot10^{-7}$	0,3977

Таблиця обґрунтовує вимогу методички не використовувати нормальні рівняння: без нормалізації число обумовленості росте від $2{,}4\cdot10^{2}$ (m = 3) до практично виродженої матриці (m ≥ 12), похибка нормальних рівнянь при m = 12 сягає 0,004, а при m = 15 — 0,4 (тобто результат непридатний), тоді як QR-розклад навіть без нормалізації помиляється не більше ніж на 10^{-6}. Нормалізація $x \to [-1; 1]$ зменшує $cond_2(A)$ на 2–5 порядків.

5.3. Графіки: 3 для Лагранжа і 3 для МНК

Рисунок 6 – Інтерполяція Лагранжа, 5 точок (поліном 4-го степеня)

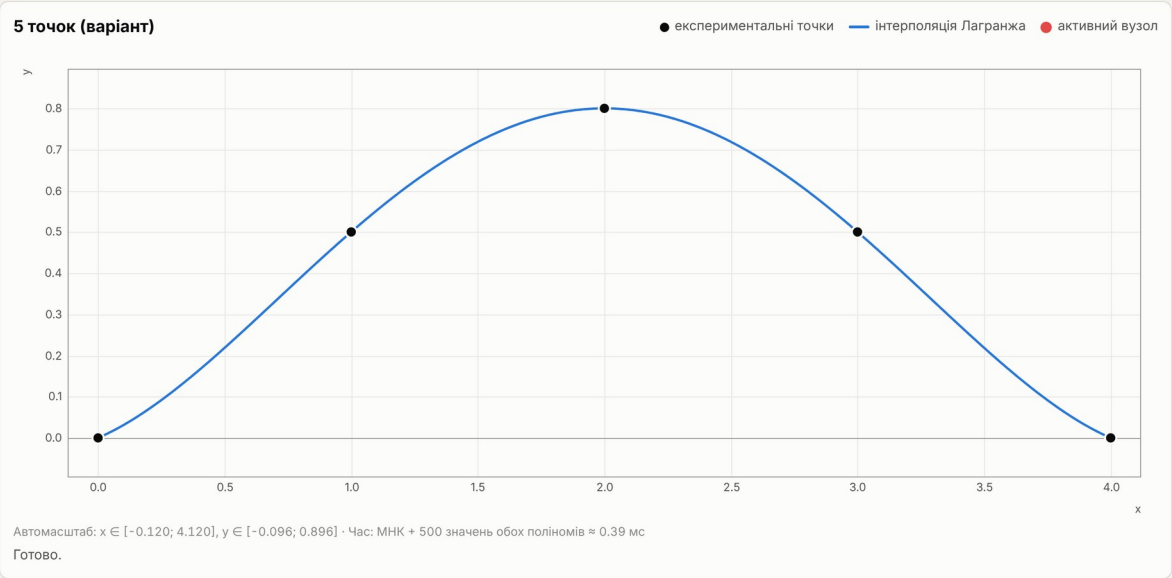


Рисунок 7 – Інтерполяція Лагранжа, 10 точок (поліном 9-го степеня)

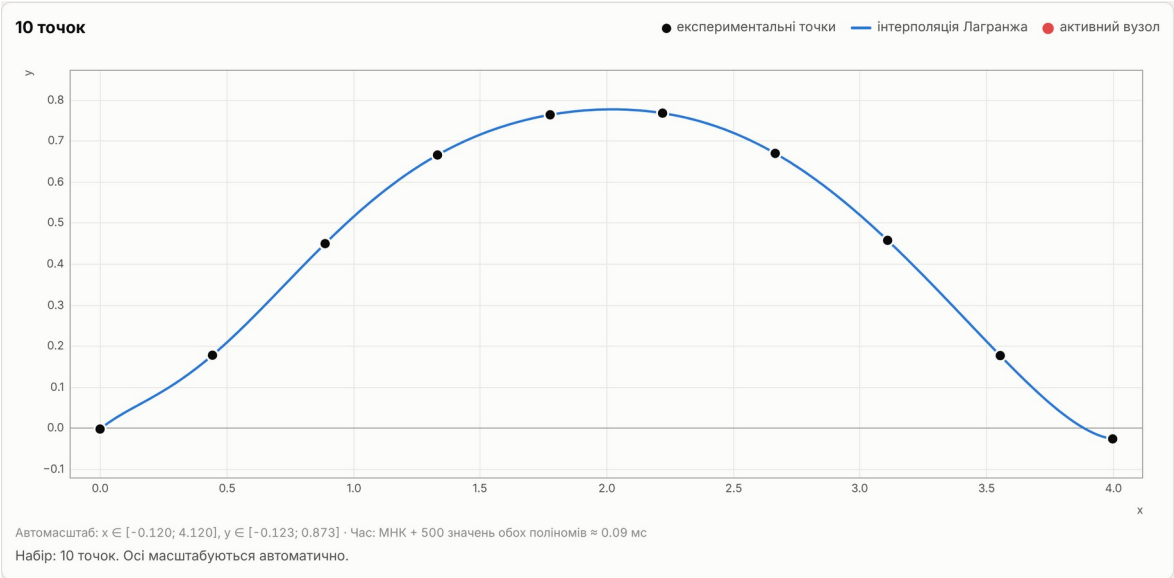


Рисунок 8 – Інтерполяція Лагранжа, 20 точок: автомасштаб охоплює осциляцію Рунге до y ≈ 23

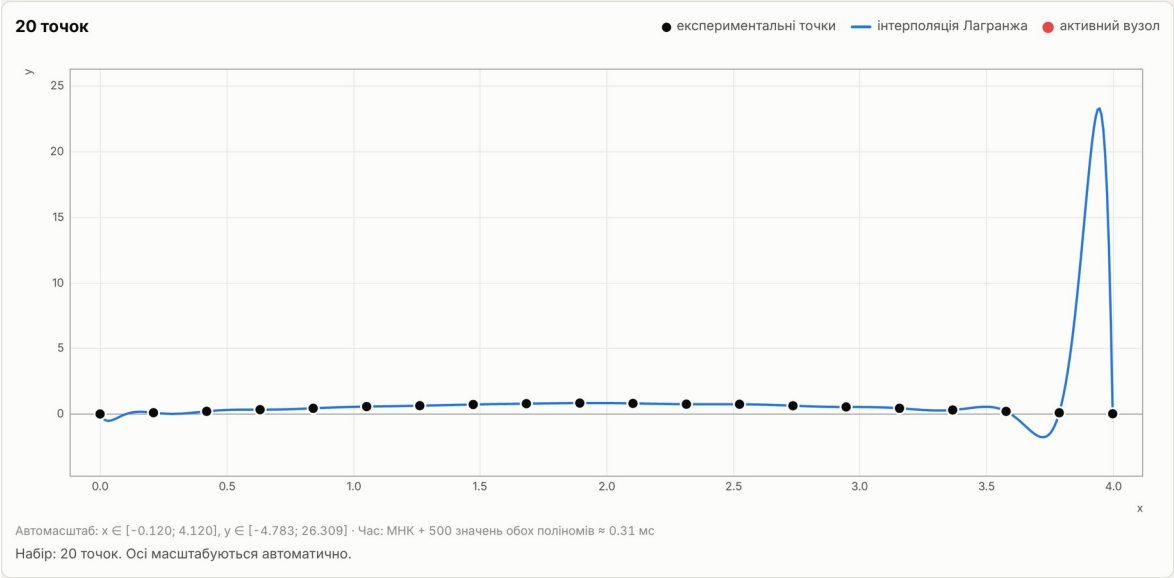


Рисунок 9 – МНК (m = 2), 5 точок, з діаграмою залишків

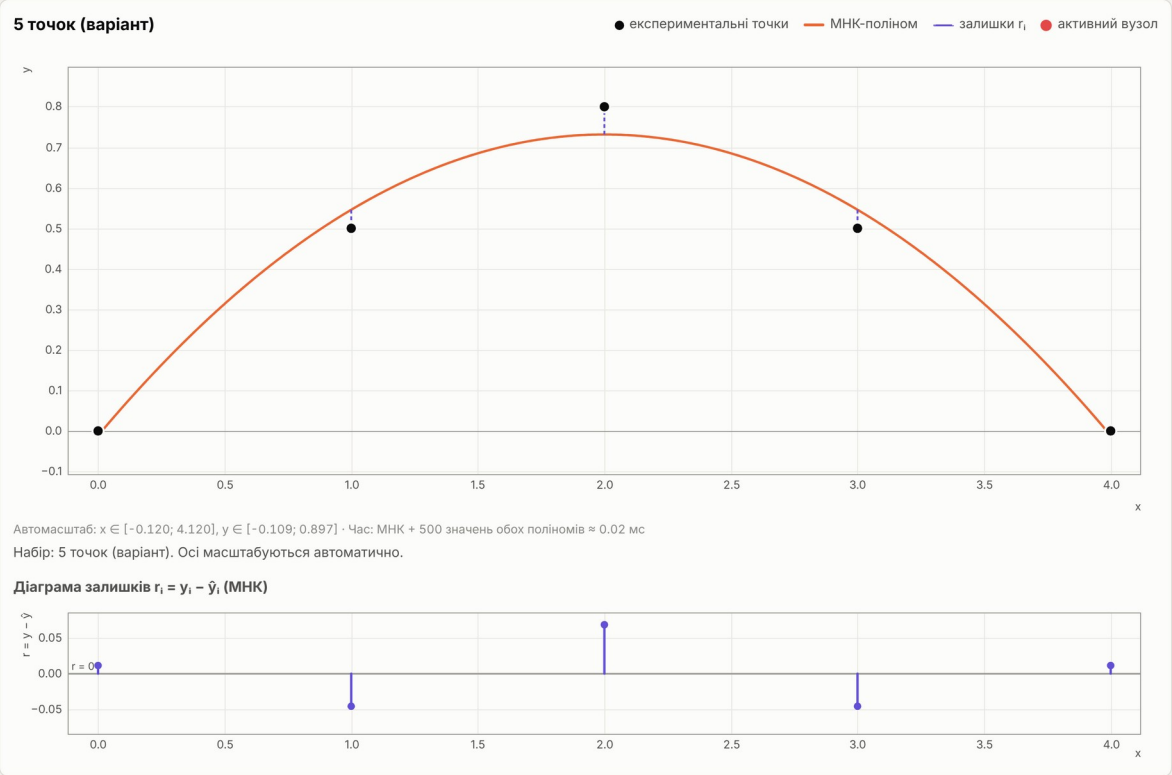


Рисунок 10 – МНК (m = 2), 10 точок, з діаграмою залишків

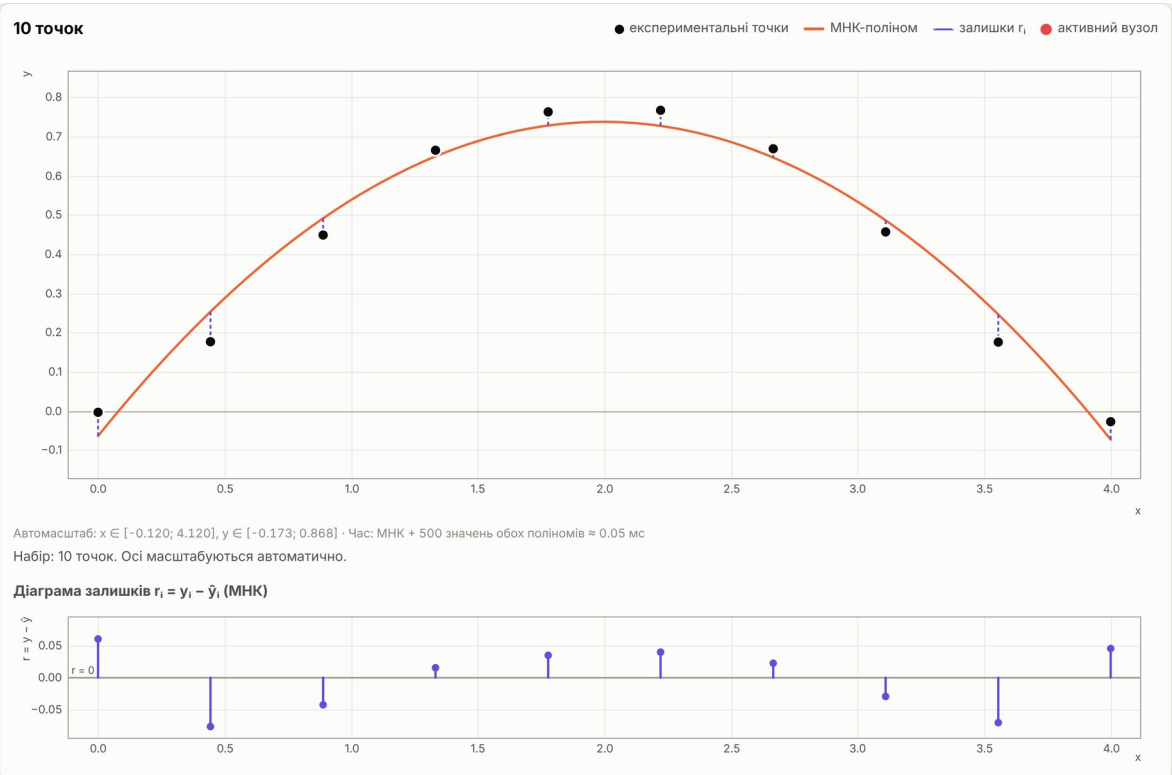


Рисунок 11 – МНК (m = 2), 20 точок, з діаграмою залишків

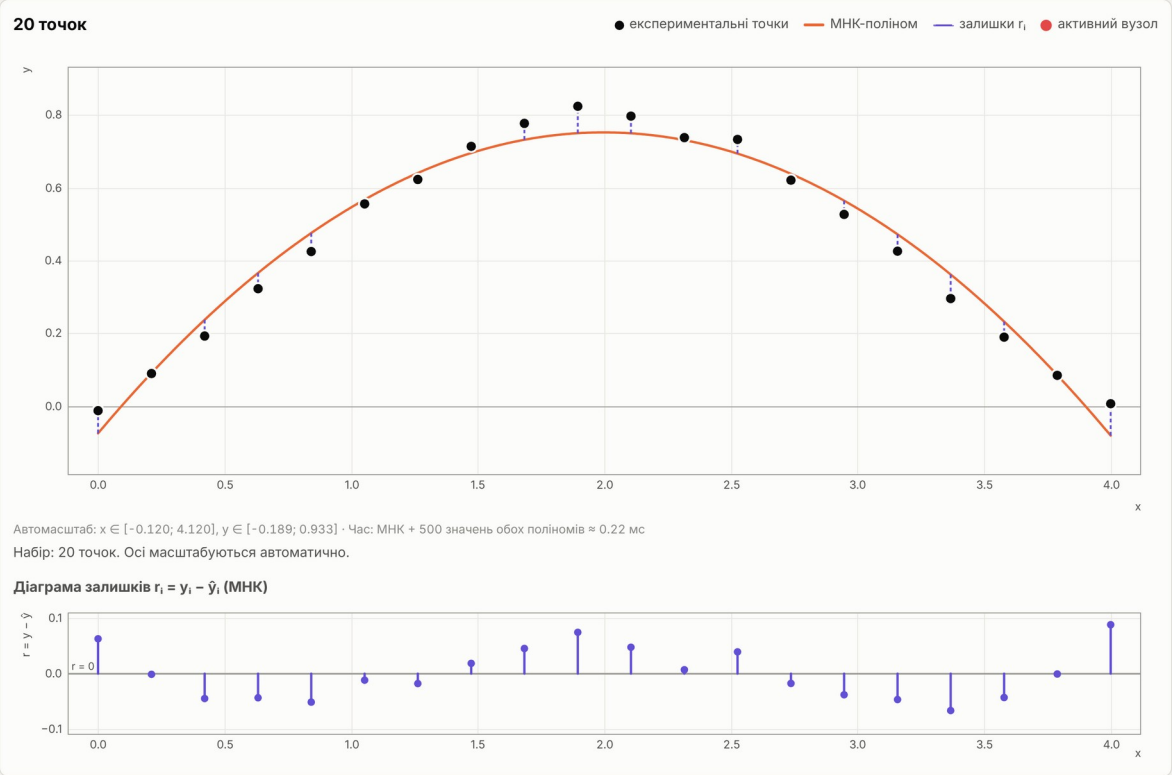
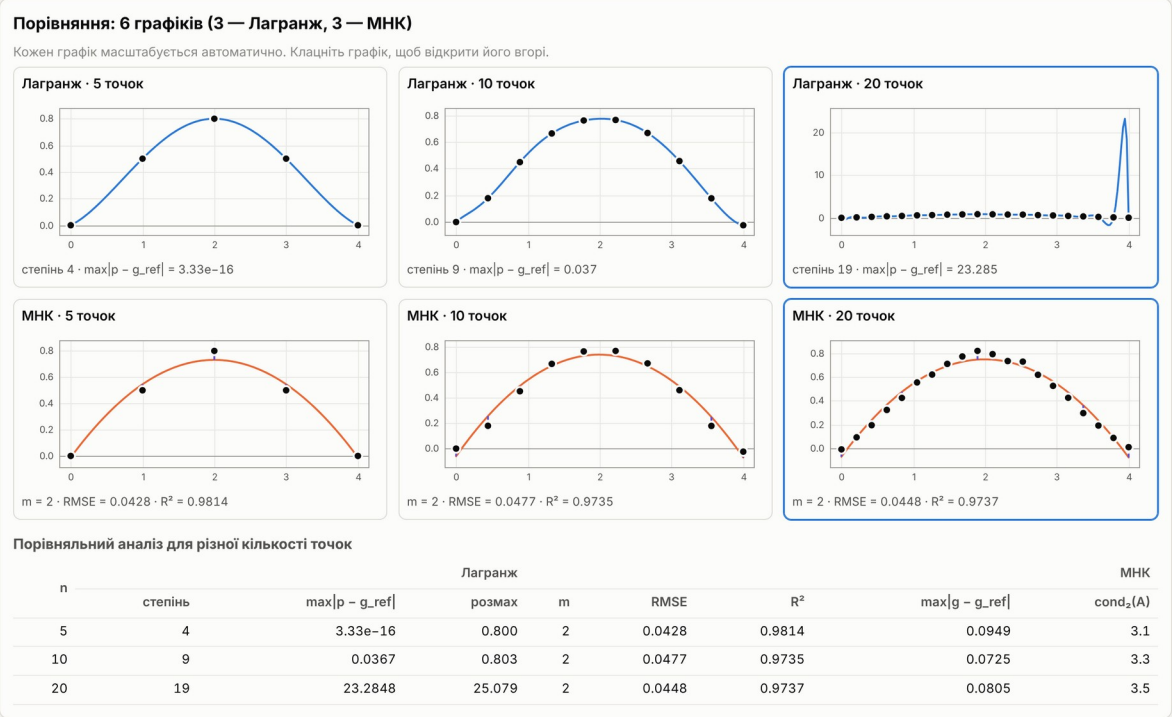


Рисунок 12 – Панель порівняння: 6 графіків з незалежним автомасштабом і порівняльна таблиця



Порівняльний аналіз для різної кількості точок

n	Лагранж					МНК	
	степінь	max|p − g_ref|	розмах	m	RMSE	R^2	max|g − g_ref|	$cond_2(A)$
5	4	3.33e−16	0.800	2	0.0428	0.9814	0.0949	3.1
10	9	0.0367	0.803	2	0.0477	0.9735	0.0725	3.3
20	19	23.2848	25.079	2	0.0448	0.9737	0.0805	3.5

5.4. Анімація, автомасштаб і підказки

Рисунок 13 – Анімація Лагранжа (10 точок): додано 3 вузли, крива тягнеться до 4-го (активний — червоний)

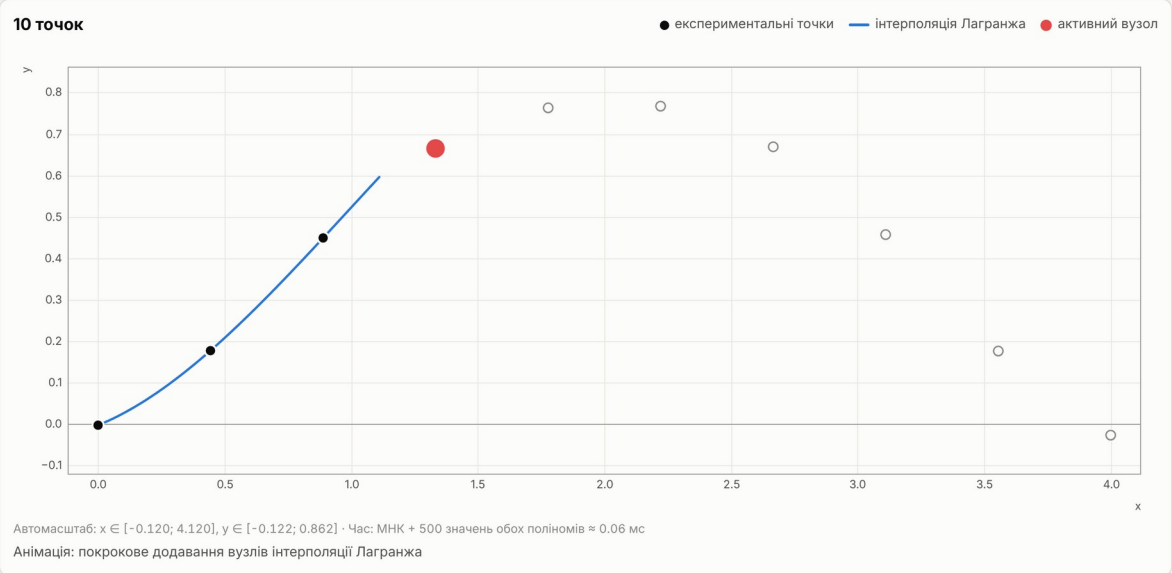


Рисунок 14 – Анімація Лагранжа (10 точок): додано 6 вузлів

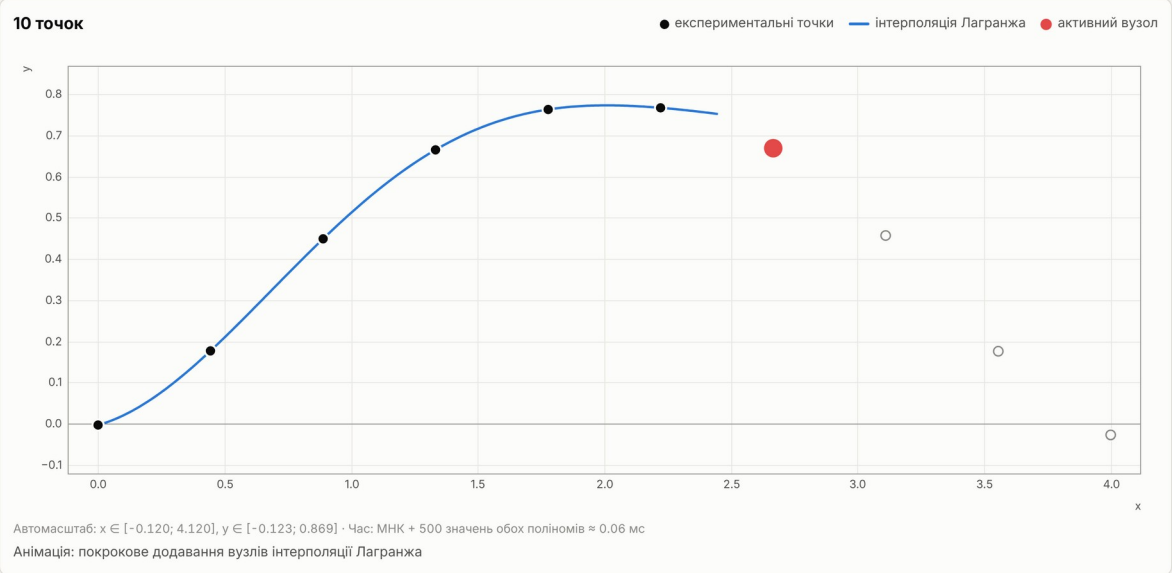


Рисунок 15 – Анімація МНК (10 точок): крива «виростає» від горизонталі ȳ

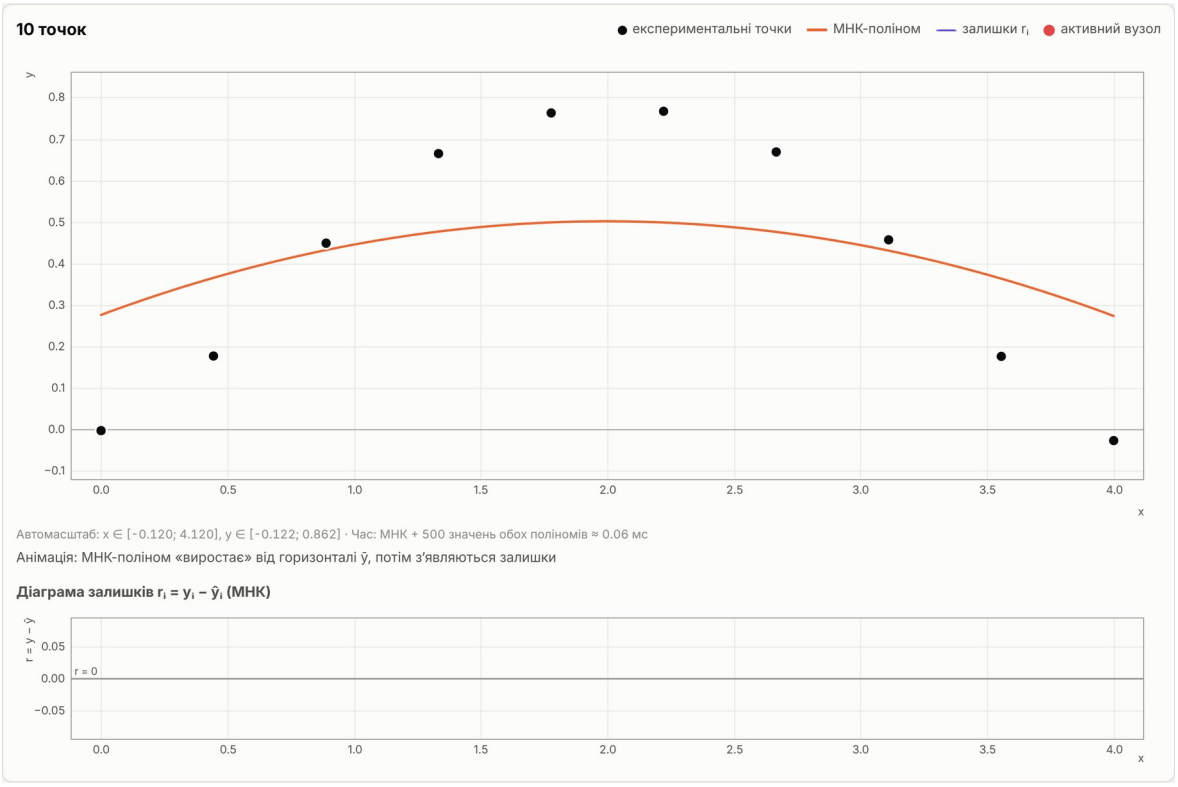


Рисунок 16 – Анімація МНК (10 точок): поступова поява залишків на графіку і на діаграмі

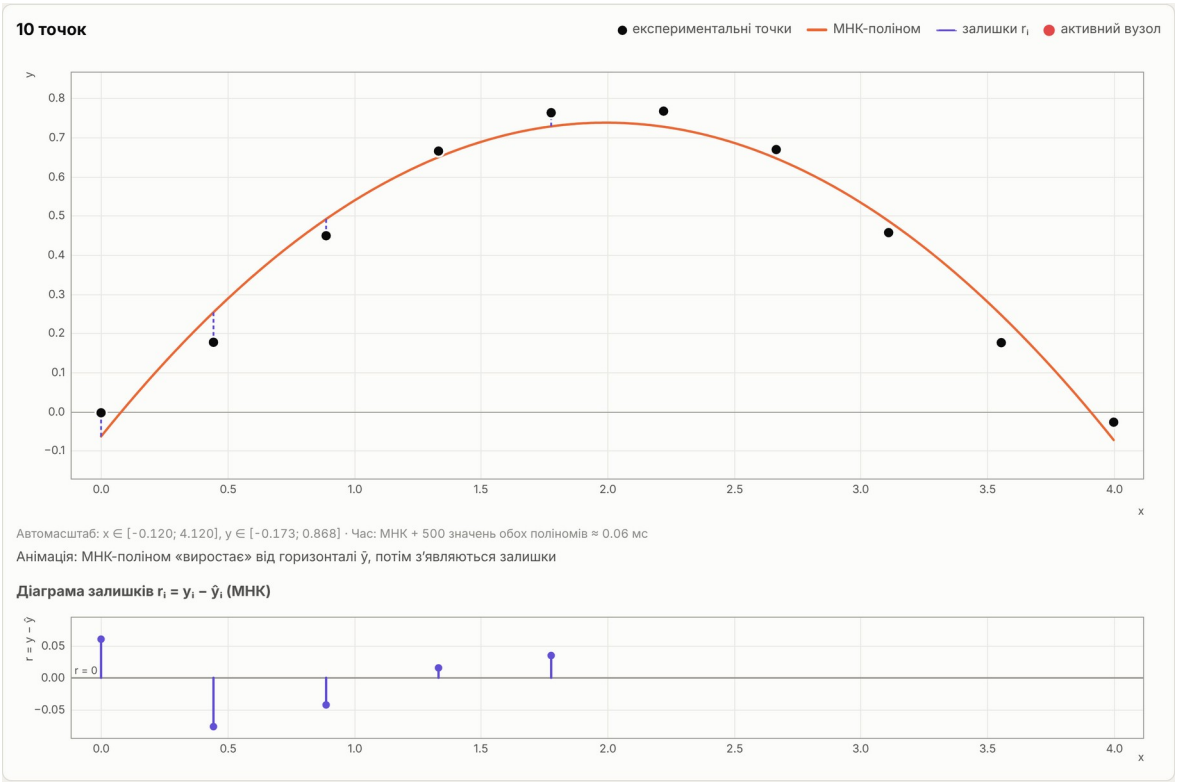


Рисунок 17 – Автомасштаб під час анімації 20 точок: межі осей підлаштовуються під проміжний поліном

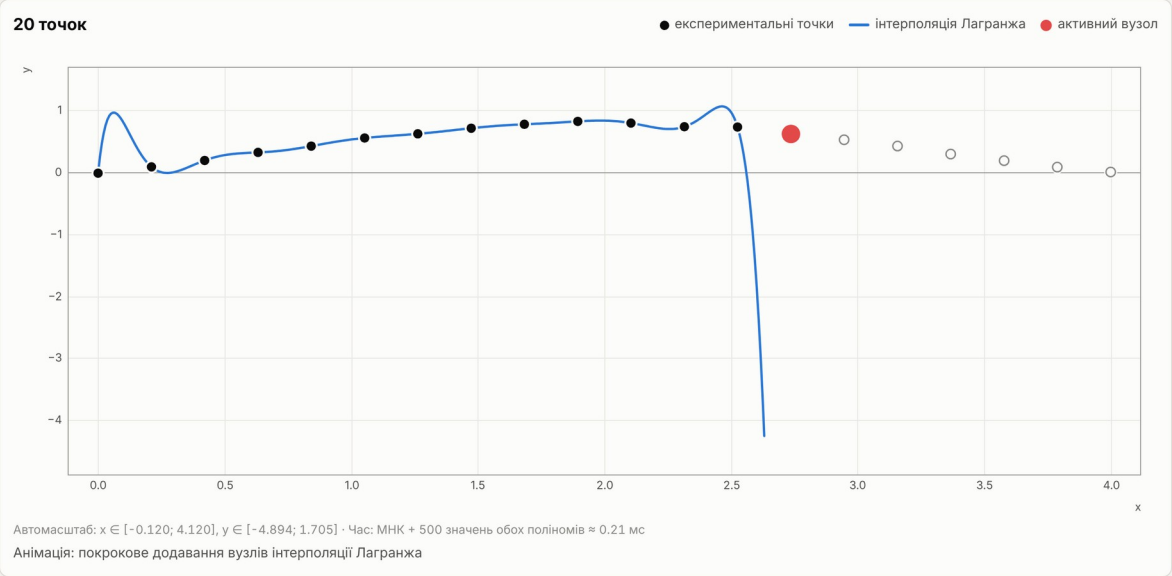


Рисунок 18 – Режим «Усі», 10 точок: підказка над вузлом (координати, $p(x_i)$, $\hat{y}_i$, залишок r_i)

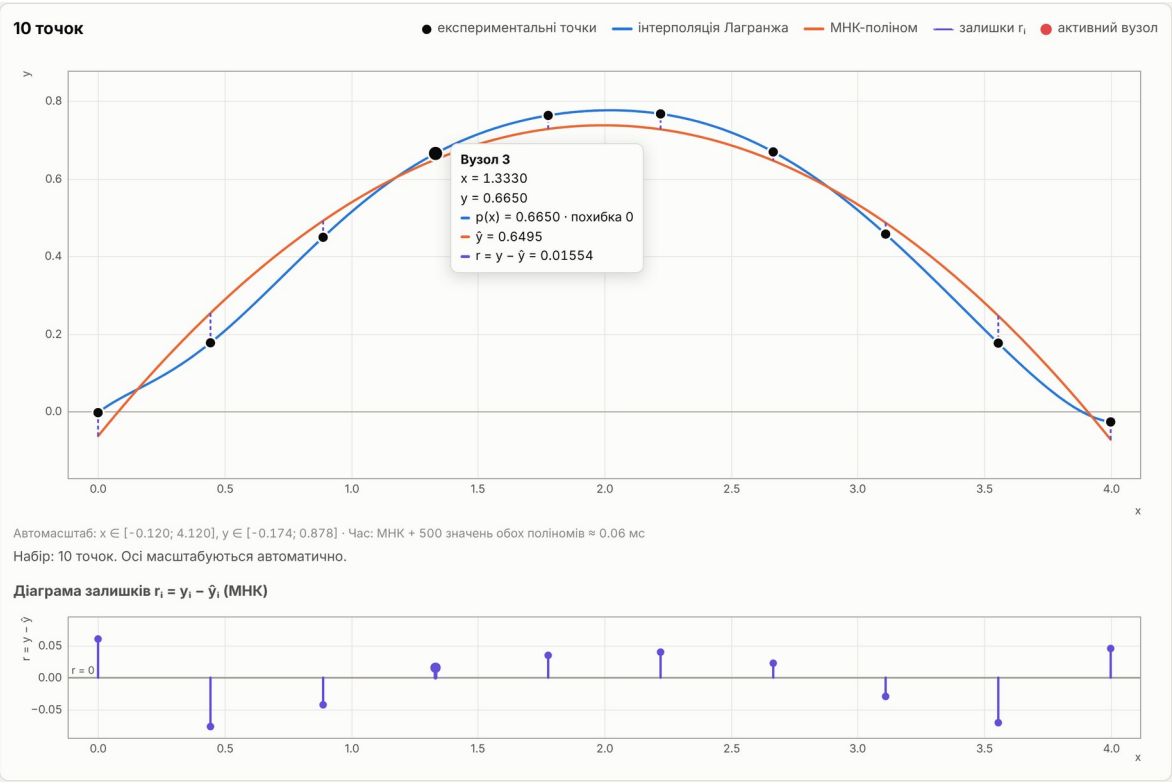
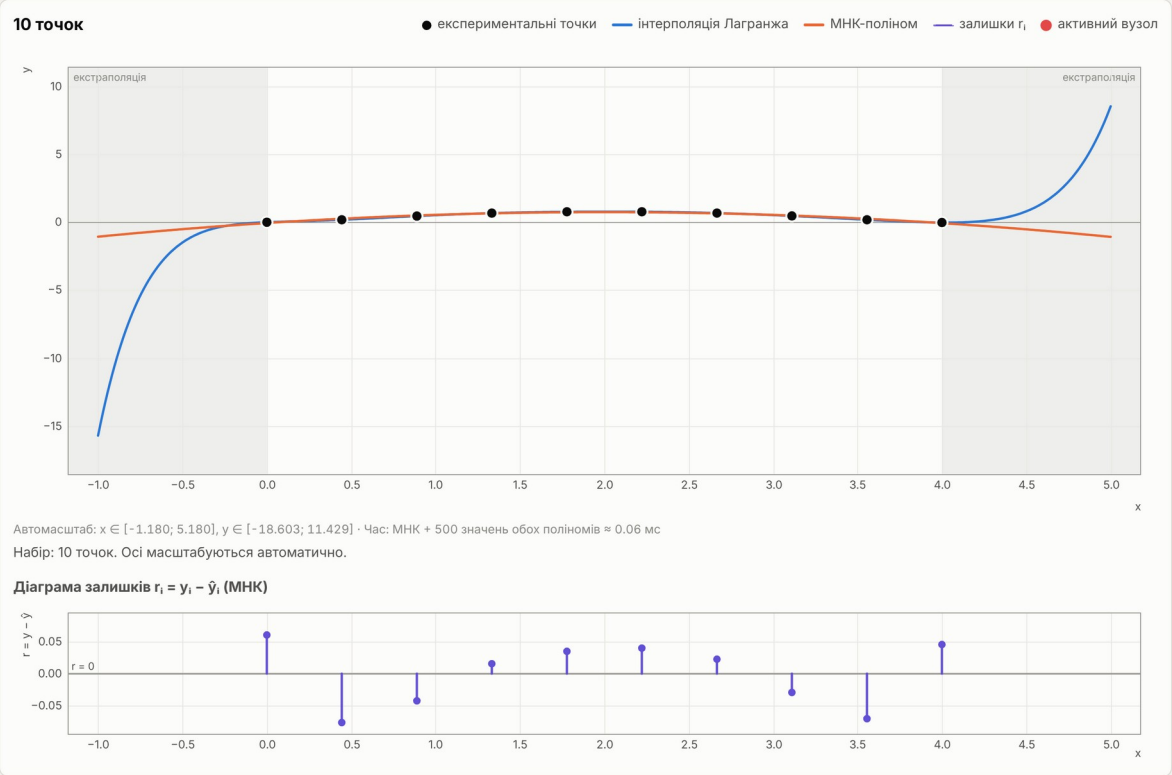


Рисунок 19 – Зона екстраполяції ±25 % (10 точок): поліноми швидко розходяться за межами вузлів



Під час анімації (рисунки 13–17) межі осей у кожному кадрі обчислюються заново по всіх видимих об'єктах: для 20 точок проміжні поліноми Лагранжа біля останнього доданого вузла виходять далеко за межі даних, і вісь Y автоматично розширюється, а після стабілізації кривої — плавно звужується. Відомості про поточні межі виводяться під графіком («Автомасштаб: x ∈ […], y ∈ […]»). Перемикання 5/10/20 відбувається миттєво: модель перераховується менш ніж за мілісекунду, осі плавно переходять до нових меж приблизно за 0,3 с.

6. Моделювання взаємодії

Програма складається з п'яти браузерних модулів. Контролер app.js отримує події інтерфейсу, бере дані з datasets.js, викликає обчислення approx.js, отримує межі осей від scale.js і передає команди малювання рендереру chart.js (клас Plot). Тести й скрипти аналізу використовують ті самі модулі в Node.js або в headless-браузері.

Рисунок 20 – Діаграма компонентів програмного продукту

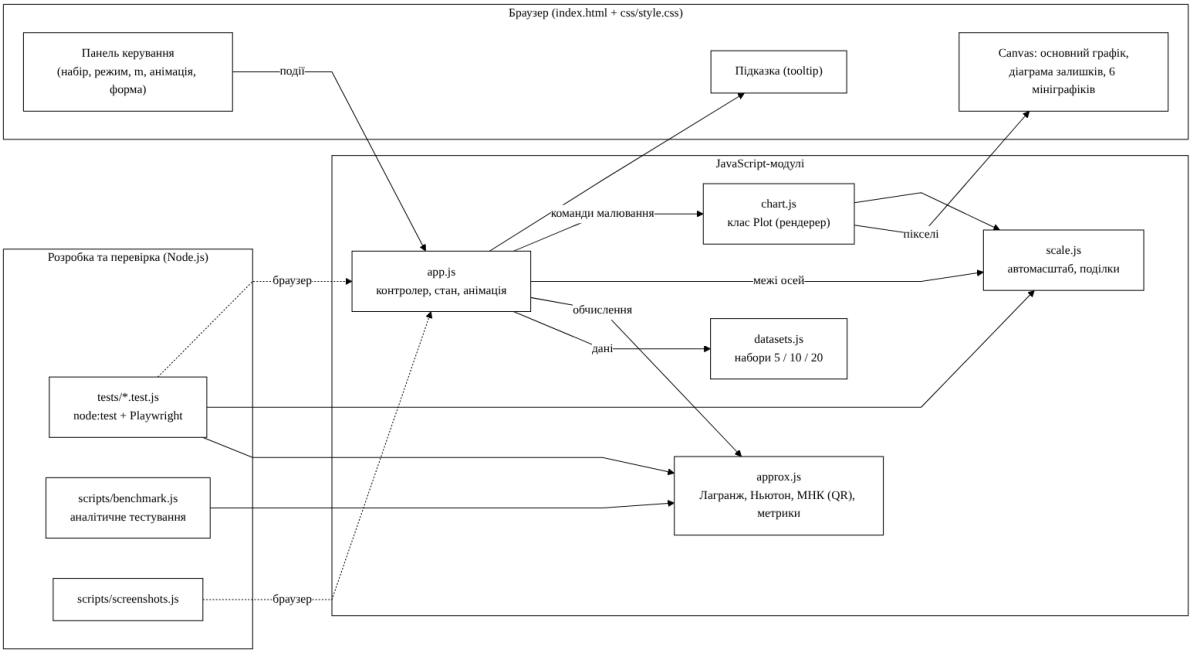


Рисунок 21 – Діаграма послідовності: перемикання набору точок (5 → 20)

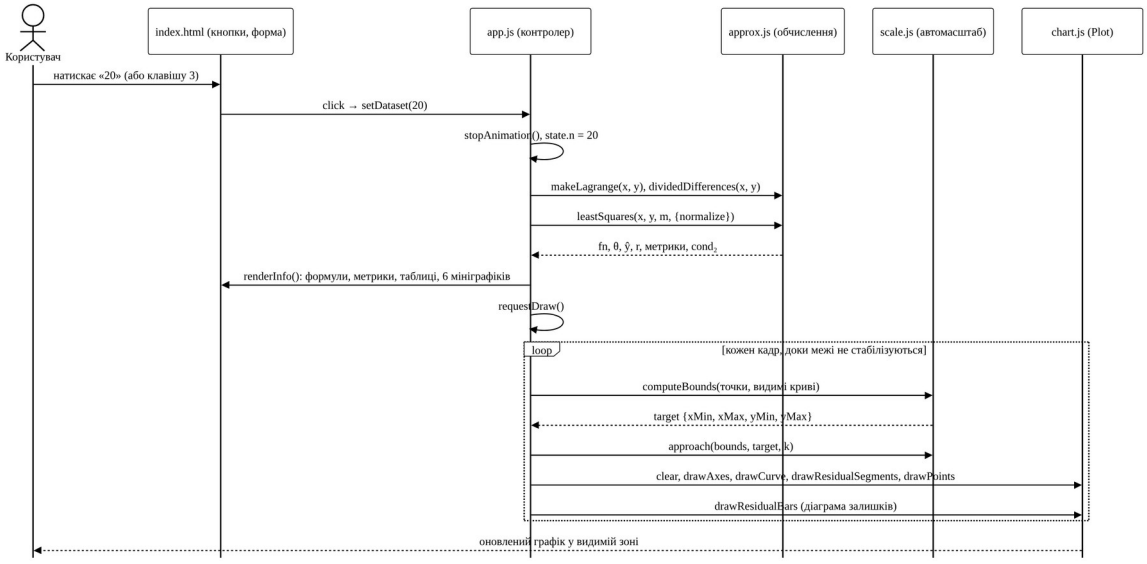
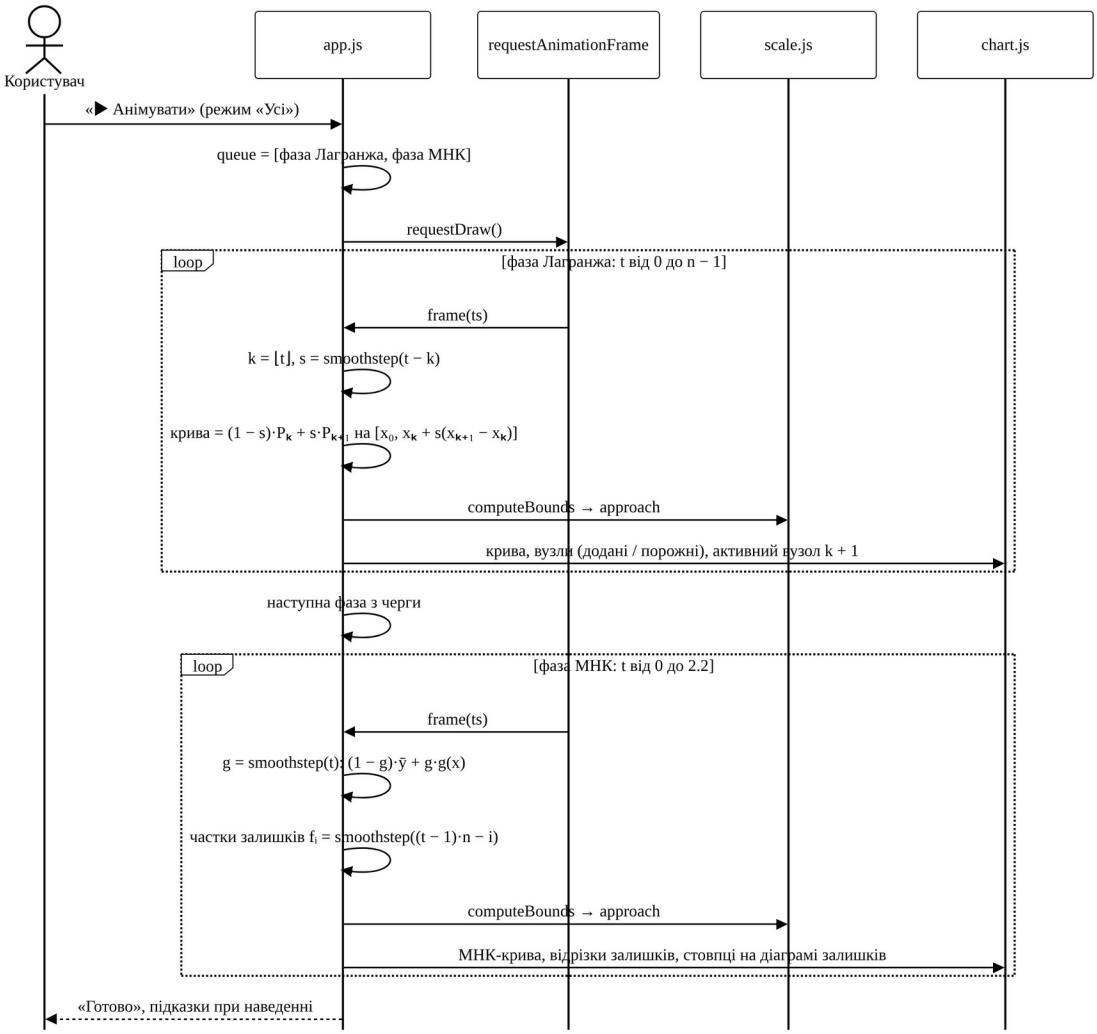


Рисунок 22 – Діаграма послідовності: анімація в режимі «Усі»



7. Документування та репозиторій GitHub

Код і документацію оформлено як git-репозиторій. Кожен логічний етап — окремий коміт із префіксом фази V-моделі та розгорнутим описом; у комітах зазначено співавторство AI (рядок Co-Authored-By). Посилання на репозиторій: https://github.com/<ваш-логін>/lab1-approximation — вставити після публікації.

Таблиця 13 – Історія комітів за фазами V-моделі

№	Коміт	Повідомлення (перший рядок)
1	313e46e	[V1 Вимоги] Технічне завдання: функціональні вимоги, інтерфейс, обмеження
2	1703720	[V1 Вимоги] Набори даних 5/10/20 точок
3	517248c	[V2 Архітектура] Вибір методології (V-модель), діаграми компонентів і взаємодії
4	55dcceb	[V3 Проєктування] Блок-схеми алгоритмів і таблиці ідентифікаторів
5	af657a4	[V4 Кодування] Математичне ядро approx.js

№	Коміт	Повідомлення (перший рядок)
6	d7d1b2f	[V4 Кодування] Автомасштабування осей і рендерер Canvas
7	8119c20	[V4 Кодування] Інтерфейс, анімації та 6 графіків порівняння
8	c866eb1	[V5 Модульні тести] Тести ядра і автомасштабу (node:test)
9	05eb953	[V6 Інтеграційні тести] UI-тести в headless-браузері (Playwright)
10	8736d86	[V7 Приймання] Аналітичне тестування 5/10/20 точок і скріншоти
11	f161f4f	[Документація] README, журнал розробки, AI Interaction Log

Склад репозиторію: README.md (опис, запуск, структура, таблиця фаз), docs/TZ.md (технічне завдання), docs/methodology.md, docs/identifiers.md, docs/diagrams (вихідні .mmd і .png), docs/analysis.md і .json (результати бенчмарку), docs/screenshots, docs/dev-notes.md (журнал знайдених помилок), docs/AI_LOG.md і docs/AI_contribution.xlsx (пункт 8).

Публікація на GitHub: створити порожній репозиторій lab1-approximation, потім у теці проєкту виконати git remote add origin https://github.com/<логін>/lab1-approximation.git і git push -u origin main.

8. Звіт про використання AI (AI Interaction Log)

8.1. Декларація ролей

Робота виконувалася з активним залученням AI-асистента Claude (Anthropic) у чаті claude.ai; **рівень залучення AI — високий**. AI запропонував ТЗ і вибір V-моделі, згенерував набори на 10 і 20 точок, реалізував код (Лагранж, Ньютон, МНК через QR Хаусхолдера, автомасштаб, анімації, інтерфейс), тести, блок-схеми й діаграми взаємодії, провів аналітичне тестування і підготував чернетку звіту. Студент поставив задачу (методичка, варіант, вимоги до мови й функцій), перевіряв роботу програми та результати, публікує репозиторій і захищає роботу. Доповніть власним внеском: які частини ви змінили, перевірили вручну чи переписали самостійно.

8.2. Найзначущіші промти

Таблиця 14 – Перелік значущих промтів

№	Промт	Для чого	Результат
1	«мені потрібно виконати цю лабораторну роботу: Розробити програмний продукт відповідно до варіанту завдання лабораторної роботи № 1. Алгоритм виконання: 1. Розробка ТЗ… 8. Звіт про використання AI… Примітка: ПЗ повинно реалізувати обчислення для 5, 10 та 20 точок… ось мій варіант: (0, 0.0), (1, 0.5), (2, 0.8), (3, 0.5), (4, 0.0)» + файл методички ЛР № 1	постановка всієї задачі	ТЗ, код, тести, діаграми, звіт
2	Додайте свої наступні промти (уточнення, виправлення, питання).		

8.3. Скріншоти діалогу та публічне посилання

Рисунок 23 – Фрагмент діалогу з AI: постановка задачі

Вставте скріншот свого першого повідомлення з файлом методички та варіантом

Рисунок 24 – Фрагмент діалогу з AI: результат і підсумок

Вставте скріншот відповіді AI з файлами результату

Публічне посилання на чат: вставити посилання (у чаті: Share → Create public link).

8.4. Критичний аналіз помилок AI

Під час розробки AI тричі помилився. Усі три помилки виявлено тестами або перевіркою в браузері, а не вигадано для звіту; їх задокументовано в docs/dev-notes.md.

Випадок 1. Хибна оцінка обумовленості. У першій версії leastSquares AI повертав як «оцінку обумовленості» відношення $\max|R_{ii}|/\min|R_{ii}|$ діагоналі матриці R. Модульний тест T-12 (нормалізація має значно покращувати обумовленість) впав: без нормалізації величина дорівнювала 1,49, а з нормалізацією — 43, тобто показувала протилежне до дійсності. Причина: діагональ R показує лише відстань кожного стовпця до попередніх, а не відношення сингулярних чисел. Виправлення: функція conditionNumber(A)

обчислює $\mathrm{cond}_2(A) = \sigma_{max}/\sigma_{min}$ через власні значення A^TA методом обертань Якобі, додано тест Т-16 на матрицях з відомим cond. Після виправлення для m = 6, n = 20 маємо $1{,}7 \cdot 10^5$ без нормалізації проти 92 з нормалізацією.

Випадок 2. Хибні еталони в тестах. У тестах Т-03 і Т-08 AI вписав неправильні «ручні» значення: p(1,5) = 0,7354 замість 0,71875 і коефіцієнти квадратичної МНК-моделі −1/35, 47/70, −5/28 замість −2/175, 26/35, −13/70. Перерахунок вручну через симетрію даних (заміна $s = (x - 2)^2$, див. п. 3.1) підтвердив, що помилялися тести, а не код. Висновок: еталонні значення від AI теж треба перевіряти незалежним розрахунком.

Випадок 3. Колізія селекторів в інтерфейсі. Обробники кнопок режиму AI навісив через querySelectorAll('[data-mode]'), але атрибут data-mode має й елемент <body> (він перемикає легенду через CSS). Через це будь-яке клацання на сторінці викликало setMode('all') і зупиняло анімацію одразу після натискання «Анімувати». Помилку виявлено під час перевірки в headless-браузері: через 0,5 с після старту state.anim = null, а стек виклику вказав на HTMLBodyElement. Виправлення: селектори button[data-mode] і button[data-n] (атрибут data-n мають і мініграфіки) та регресійний тест UI-04.

8.5. Таблиця внеску

Розподіл внеску за етапами наведено у файлі docs/AI_contribution.xlsx (аркуші «Внесок», «Промти», «Помилки AI»). Внесок студента вводиться в жовті клітинки, а внесок AI та зважений підсумок рахуються формулами. Значення в таблиці 15 — оцінка за фактичним ходом роботи; уточніть їх за своїм реальним внеском.

Таблиця 15 – Внесок студента та AI за етапами (з файлу AI_contribution.xlsx)

№	Етап	Роль студента	Роль AI	Студент	AI
1	Технічне завдання	Надав методичку, варіант і вимоги до програми; погодив ТЗ	Сформулював ТЗ за методичкою, таблиця «вимога → тест»	15%	85%
2	Вибір методології	Погодив вибір	Обґрунтував V-модель, склав таблицю фаз	10%	90%
3	Проєктування логіки	Перевірка відповідності коду	Побудував блок-схеми і таблиці	5%	95%
4	Програмна реалізація	Запуск і перевірка в браузері	Написав код js/*.js, index.html, css	5%	95%

№	Етап	Роль студента	Роль AI	Студент	AI
5	Аналітичне тестування	Запуск npm test / npm run bench, аналіз результатів	Написав тести і бенчмарк, знайшов і виправив 3 помилки	10%	90%
6	Моделювання взаємодії	Перевірка	Побудував діаграми	5%	95%
7	Документування і GitHub	Створення репозиторію на GitHub, push, перевірка звіту	Підготував README, локальний git з комітами, звіт	30%	70%
8	AI Interaction Log	Скріншоти діалогу, публічне посилання, власна декларація	Чернетка розділу, опис фактичних помилок	50%	50%
	Разом (зважено за вагою етапів)			**12%**	**88%**

ВИСНОВКИ

У лабораторній роботі розроблено веб-застосунок на JavaScript для апроксимації табличних даних інтерполяційним поліномом Лагранжа (з таблицею поділених різниць Ньютона) і методом найменших квадратів. МНК реалізовано через QR-розклад Хаусхолдера з нормалізацією змінної, без нормальних рівнянь. Програма виконує обчислення для 5, 10 і 20 точок, будує 6 графіків (3 для Лагранжа і 3 для МНК), анімує обидва методи, автоматично масштабує осі X і Y за обчисленими min і max видимих об'єктів у кожному кадрі та миттєво перемикає набори.

Розробку проведено за V-моделлю: кожній фазі специфікації (ТЗ, архітектура, детальне проєктування) відповідає рівень перевірки (приймальне, інтеграційне, модульне тестування). Усі 30 автоматизованих тестів проходять; кожна фаза зафіксована окремим комітом.

Аналітичне тестування показало, що глобальна інтерполяція на рівновіддалених вузлах із шумом стає непридатною зі зростанням n: для 20 точок поліном 19-го степеня осцилює до $y \approx 23{,}3$ при даних до 0,82. Водночас МНК з $m = 2$ стабільно дає RMSE ≈ 0,045 і $R^2 \approx 0{,}97$, а з $m = 4$ — RMSE ≈ 0,012. Отже, для експериментальних даних з похибками доцільніше використовувати МНК невисокого степеня, а інтерполяцію — для точних даних і невеликої кількості вузлів. Нормальні рівняння при великих m

втрачають точність до 0,4, тоді як QR-розклад залишається точним, а нормалізація x зменшує обумовленість на кілька порядків.

AI суттєво пришвидшив роботу, але тричі помилився (хибна оцінка обумовленості, хибні еталони в тестах, колізія селекторів). Усі помилки виявили автоматизовані тести та перевірка в браузері, тому результати AI обов'язково треба перевіряти незалежними розрахунками й тестами.

ПЕРЕЛІК ДЖЕРЕЛ

1. Методичні вказівки до лабораторної роботи № 1 «Апроксимація функцій методами інтерполяції та МНК». – Ужгород, 2026.

2. Golub G. H., Van Loan C. F. Matrix Computations. 4th ed. – Baltimore : Johns Hopkins University Press, 2013. – 756 p.

3. Trefethen L. N., Bau D. Numerical Linear Algebra. – Philadelphia : SIAM, 1997. – 361 p.

4. Burden R. L., Faires J. D. Numerical Analysis. 9th ed. – Boston : Brooks/Cole, 2011. – 872 p.

5. MDN Web Docs. Canvas API. – URL: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

6. MDN Web Docs. Window: requestAnimationFrame() method. – URL: https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame

7. Node.js Documentation. Test runner (node:test). – URL: https://nodejs.org/api/test.html

8. Playwright Documentation. – URL: https://playwright.dev/docs/intro

9. Mermaid Documentation: Flowcharts, Sequence diagrams. – URL: https://mermaid.js.org/

ДОДАТОК А

Відповіді на контрольні запитання

1. У чому різниця між інтерполяцією та апроксимацією МНК для табличних даних?

Інтерполяційна функція проходить точно через усі вузли: $g(x_i) = y_i$. МНК-функція не зобов'язана проходити через точки, вона мінімізує суму квадратів відхилень $\Sigma(y_i - g(x_i))^2$. Інтерполяція доречна для точних даних і невеликої кількості вузлів; МНК — для експериментальних даних з похибками і великої кількості точок, бо згладжує шум. У роботі це видно на 20 точках: поліном Лагранжа 19-го степеня осцилює до $y \approx 23$, а МНК $m = 2$ має RMSE ≈ 0,045.

2. У чому полягає різниця між інтерполяцією та екстраполяцією? Наведіть приклади практичного застосування.

Інтерполяція — обчислення значень усередині відрізка вузлів $[x_0; x_n]$, екстраполяція — за його межами. Інтерполяція: значення таблиць (густина, теплоємність) для проміжної температури, відновлення пропущених вимірювань, плавні траєкторії в комп'ютерній графіці. Екстраполяція: прогноз продажів чи навантаження на наступний період, оцінка значення фізичної величини поза межами експерименту. Екстраполяція значно ризикованіша, бо поза вузлами модель нічим не обмежена.

3. Як реалізувати інтерполяцію Лагранжа мовою JavaScript? Які труднощі можуть виникнути?

Подвійний цикл: зовнішній за k накопичує $s \mathrel{+}= y_k \cdot L_k(x)$, внутрішній за $j \neq k$ обчислює добуток $(x - x_j)/(x_k - x_j)$ (функція lagrange у js/approx.js). Труднощі: ділення на нуль при однакових вузлах (потрібна перевірка validatePoints); складність $O(n^2)$ на кожне значення (для кривої з сотень точок краще форма Ньютона або барицентрична форма); ефект Рунге для високих степенів на рівновіддалених вузлах; похибки округлення у числах типу

Number (IEEE 754); значення поза відрізком (екстраполяція) можуть бути дуже великими, тож графік потребує автомасштабування й обрізання.

4. Як можна використати методи екстраполяції для прогнозування у прикладних задачах?

Будують модель за наявними даними (зазвичай МНК невисокого степеня або модель, обґрунтовану фізикою/економікою: лінійний тренд, експонента) і обчислюють її значення для майбутніх моментів. Наприклад, тренд продажів y = a + bt за кварталами дає прогноз на наступний квартал; закон охолодження — прогноз температури. Прогноз роблять на невеликий горизонт, оцінюють похибку (залишки, перехресна перевірка) і не використовують поліноми високого степеня, бо вони за межами вузлів швидко розходяться (рисунок 19).

5. Реалізуйте програмний код, що будує інтерполяційний поліном Ньютона та обчислює значення в довільній точці.

Реалізовано у функціях dividedDifferences і newtonEval (js/approx.js), лістинг нижче. Для варіанта коефіцієнти a = [0; 0,5; −0,1; −1/15; 1/30], і p(2,5) = 0,8 − 0,25/3 + 0,0625/30 ≈ 0,71875 збігається з Лагранжем (тест T-05).

```
function dividedDifferences(xs, ys) {
  const n = xs.length, table = [ys.slice()];
  for (let k = 1; k < n; k++) {
    const prev = table[k - 1], col = [];
    for (let i = 0; i < n - k; i++)
      col.push((prev[i + 1] - prev[i]) / (xs[i + k] - xs[i]));
    table.push(col);
  }
  return { table, coeffs: table.map((col) => col[0]) };
}
function newtonEval(xs, coeffs, x) {      // схема Горнера
  let v = coeffs[coeffs.length - 1];
  for (let k = coeffs.length - 2; k >= 0; k--) v = v * (x - xs[k]) +
coeffs[k];
  return v;
}
const { coeffs } = dividedDifferences([0,1,2,3,4], [0,0.5,0.8,0.5,0]);
console.log(newtonEval([0,1,2,3,4], coeffs, 2.5)); // 0.71875
```

6. Які елементи HTML та прийоми CSS зручно використати для побудови та анімації графіків?

<canvas> для швидкого малювання кривих (у роботі — Canvas 2D з урахуванням devicePixelRatio) або <svg> для векторної графіки з подіями на елементах; кілька полотен або абсолютно позиційованих шарів (графік, підказка-div поверх полотна); CSS custom properties для кольорів кривих і тем (світла/темна), clamp() і відсоткова ширина для адаптивності, grid/flex для панелей і сітки мініграфіків; ResizeObserver для перемальовування при зміні розміру; виділення точок і залишків — зміною радіуса/кольору маркера та пунктирними відрізками.

7. Які події або механізми у JavaScript доцільно використати для покрокової анімації?

requestAnimationFrame — синхронізований з частотою екрана цикл кадрів (у роботі функція frame); параметр часу t нарощується на dt·швидкість, тому анімація не залежить від FPS; черга фаз (queue: спершу Лагранж, потім МНК); функції згладжування (smoothstep) для плавності; setTimeout/setInterval менш точні й не зупиняються у фонових вкладках; події click/keydown/input для керування (пауза, швидкість, перемикання наборів), mousemove для підказок.

8. Як впливають масштабування та нормалізація змінної x на числову стійкість поліномів?

Стовпці матриці Вандермонда 1, x, x^2, … для $x \in [0; 4]$ мають дуже різний масштаб і майже лінійно залежні, тому cond(A) швидко зростає. Перехід до $t = (x - c)/h \in [-1; 1]$ вирівнює масштаб: для 20 точок при m = 6 $cond_2(A)$ зменшується з $1{,}7 \cdot 10^5$ до 92, при m = 9 — з $1{,}7 \cdot 10^8$ до $1{,}3 \cdot 10^3$ (таблиця 12). Результат апроксимації при цьому той самий (тест Т-12), але похибки округлення в числах подвійної точності значно менші. Додатково допомагає QR-розклад замість нормальних рівнянь, де обумовленість підноситься до квадрата.

9. Які ризики виникають під час екстраполяції і як їх візуально продемонструвати у веб-інтерфейсі?

За межами вузлів поліном нічим не обмежений: поліноми високого степеня швидко йдуть до ±∞, а різні моделі, однаково добрі всередині відрізка, дають дуже різні прогнози. У програмі прапорець «екстраполяція ±25 %» розширює відрізок побудови, зони за межами [x_0; x_n] зафарбовано сірим і підписано «екстраполяція», підказка показує |p − g| і позначку «екстраполяція», а автомасштаб робить розбіжність кривих видимою (рисунок 19).

ДОДАТОК Б

Програмний код

Лістинг: js/approx.js

```
/**
 * approx.js — математичне ядро лабораторної роботи № 1.
 *  • інтерполяційний поліном Лагранжа (формула 4.3);
 *  • поліном Ньютона з таблицею поділених різниць (формула 4.4);
 *  • МНК-апроксимація поліномом степеня m через QR-розклад Хаусхолдера
 *    (без нормальних рівнянь), з нормалізацією x → t ∈ [−1; 1];
 *  • метрики якості (SSE, RMSE, R², max|r|).
 * Працює у браузері (глобальний Approx) і в Node.js (module.exports).
 */
(function (root, factory) {
  const api = factory();
  if (typeof module === 'object' && module.exports) module.exports = api;
  else root.Approx = api;
})(typeof self !== 'undefined' ? self : this, function () {
  'use strict';

  /* ---------------------------------------------------------------- перевірки */

  /** Перевіряє набір точок: однакова довжина, скінченні числа, різні вузли. */
  function validatePoints(xs, ys) {
    if (!Array.isArray(xs) || !Array.isArray(ys)) throw new TypeError('xs та ys мають бути
масивами');
    if (xs.length !== ys.length) throw new RangeError('Кількість x і y не збігається');
    if (xs.length < 2) throw new RangeError('Потрібно щонайменше 2 точки');
    for (let i = 0; i < xs.length; i++) {
      if (!Number.isFinite(xs[i]) || !Number.isFinite(ys[i])) {
        throw new TypeError(`Точка ${i + 1}: значення мають бути скінченними числами`);
      }
    }
    const sorted = [...xs].sort((a, b) => a - b);
    for (let i = 1; i < sorted.length; i++) {
      if (sorted[i] === sorted[i - 1]) throw new RangeError(`Вузли мають бути різними (x = $
{sorted[i]} повторюється)`);
    }
  }

  /* ---------------------------------------------------------------- Лагранж */

  /** Базисний поліном L_k(x) = Π_{j≠k} (x − x_j)/(x_k − x_j). */
  function lagrangeBasis(xs, k, x) {
    let l = 1;
    for (let j = 0; j < xs.length; j++) {
      if (j !== k) l *= (x - xs[j]) / (xs[k] - xs[j]);
    }
    return l;
  }

  /** Значення інтерполяційного полінома Лагранжа p_n(x) = Σ y_k·L_k(x). */
  function lagrange(xs, ys, x) {
    let s = 0;
    for (let k = 0; k < xs.length; k++) s += ys[k] * lagrangeBasis(xs, k, x);
    return s;
  }

  /** Повертає функцію-поліном Лагранжа для набору (зручно для побудови кривих). */
  function makeLagrange(xs, ys) {
    validatePoints(xs, ys);
    const X = xs.slice();
    const Y = ys.slice();
    return (x) => lagrange(X, Y, x);
  }

  /* ---------------------------------------------------------------- Ньютон */

  /**
   * Таблиця поділених різниць: table[k][i] = f[x_i, …, x_{i+k}].
   * Коефіцієнти полінома Ньютона — верхня діагональ: a_k = table[k][0].
   */
  function dividedDifferences(xs, ys) {
    validatePoints(xs, ys);
    const n = xs.length;
```

```
    const table = [ys.slice()];
    for (let k = 1; k < n; k++) {
      const prev = table[k - 1];
      const col = [];
      for (let i = 0; i < n - k; i++) col.push((prev[i + 1] - prev[i]) / (xs[i + k] - xs[i]));
      table.push(col);
    }
    return { table, coeffs: table.map((col) => col[0]) };
  }

  /** Значення полінома Ньютона за схемою Горнера. */
  function newtonEval(xs, coeffs, x) {
    let v = coeffs[coeffs.length - 1];
    for (let k = coeffs.length - 2; k >= 0; k--) v = v * (x - xs[k]) + coeffs[k];
    return v;
  }

  /* ---------------------------------------------------------------- МНК */

  /** Параметри нормалізації: t = (x − c)/h, щоб t Є [−1; 1]. */
  function normalization(xs, enabled = true) {
    if (!enabled) return { c: 0, h: 1 };
    const lo = Math.min(...xs);
    const hi = Math.max(...xs);
    return { c: (lo + hi) / 2, h: (hi - lo) / 2 || 1 };
  }

  /** Дизайн-матриця (матриця Вандермонда) A[i][j] = t_i^j, j = 0..m. */
  function designMatrix(ts, m) {
    return ts.map((t) => {
      const row = new Array(m + 1);
      row[0] = 1;
      for (let j = 1; j <= m; j++) row[j] = row[j - 1] * t;
      return row;
    });
  }

  /**
   * Розв'язує перевизначену систему A·θ ≈ b у сенсі найменших квадратів
   * QR-розкладом Хаусхолдера: A = Q·R, θ = R⁻¹·(QᵀB).
   * Повертає θ і діагональ R (для оцінки обумовленості).
   */
  function householderLeastSquares(A, b) {
    const rows = A.length;
    const cols = A[0].length;
    if (rows < cols) throw new RangeError(`Для степеня m = ${cols - 1} потрібно щонайменше ${cols} точок`);
    const R = A.map((r) => r.slice());
    const qtb = b.slice();

    for (let k = 0; k < cols; k++) {
      // норма k-го стовпця нижче діагоналі
      let norm = 0;
      for (let i = k; i < rows; i++) norm += R[i][k] * R[i][k];
      norm = Math.sqrt(norm);
      if (norm === 0) throw new RangeError('Матриця вироджена (стовпці лінійно залежні)');
      const alpha = R[k][k] > 0 ? -norm : norm; // знак обрано так, щоб уникнути віднімання близьких чисел
      // вектор відбиття v = x − alpha·e1
      const v = new Array(rows).fill(0);
      for (let i = k; i < rows; i++) v[i] = R[i][k];
      v[k] -= alpha;
      let vv = 0;
      for (let i = k; i < rows; i++) vv += v[i] * v[i];
      if (vv === 0) continue;
      // H = I − 2·v·vᵀ/(vᵀv) застосовується до стовпців R та до b
      for (let j = k; j < cols; j++) {
        let s = 0;
        for (let i = k; i < rows; i++) s += v[i] * R[i][j];
        const f = (2 * s) / vv;
        for (let i = k; i < rows; i++) R[i][j] -= f * v[i];
      }
      let s = 0;
      for (let i = k; i < rows; i++) s += v[i] * qtb[i];
      const f = (2 * s) / vv;
      for (let i = k; i < rows; i++) qtb[i] -= f * v[i];
    }

    // зворотна підстановка R·θ = (QᵀB)[0..cols)
    const theta = new Array(cols).fill(0);
```

```
    for (let i = cols - 1; i >= 0; i--) {
      let s = qtb[i];
      for (let j = i + 1; j < cols; j++) s -= R[i][j] * theta[j];
      if (Math.abs(R[i][i]) < 1e-14) throw new RangeError('Матриця погано обумовлена або
вироджена');
      theta[i] = s / R[i][i];
    }
    return { theta };
  }

  /** Обчислює поліном Σ θ_j·t^j за Горнером. */
  function polyEval(theta, t) {
    let v = 0;
    for (let j = theta.length - 1; j >= 0; j--) v = v * t + theta[j];
    return v;
  }

  /** Перерахунок коефіцієнтів з t = (x − c)/h у звичайні коефіцієнти за степенями x. */
  function toXCoefficients(theta, c, h) {
    const m = theta.length - 1;
    const out = new Array(m + 1).fill(0);
    // ((x − c)/h)^j = Σ_k C(j,k)·x^k·(−c)^{j−k} / h^j
    for (let j = 0; j <= m; j++) {
      let binom = 1;
      for (let k = 0; k <= j; k++) {
        if (k > 0) binom = (binom * (j - k + 1)) / k;
        out[k] += (theta[j] * binom * (-c) ** (j - k)) / h ** j;
      }
    }
    return out;
  }

  /**
   * МНК-апроксимація поліномом степеня m.
   * @returns {{m, theta, coeffsX, norm, fn, fitted, residuals, metrics, cond}}
   */
  function leastSquares(xs, ys, m, { normalize = true } = {}) {
    validatePoints(xs, ys);
    if (!Number.isInteger(m) || m < 0) throw new RangeError('Степінь m має бути цілим
невід’ємним числом');
    if (m > xs.length - 1) throw new RangeError(`Для ${xs.length} точок степінь m не може
перевищувати ${xs.length - 1}`);
    const norm = normalization(xs, normalize);
    const ts = xs.map((x) => (x - norm.c) / norm.h);
    const A = designMatrix(ts, m);
    const { theta } = householderLeastSquares(A, ys);
    const fn = (x) => polyEval(theta, (x - norm.c) / norm.h);
    const fitted = xs.map(fn);
    const residuals = ys.map((y, i) => y - fitted[i]);
    return {
      m,
      theta,
      coeffsX: toXCoefficients(theta, norm.c, norm.h),
      norm,
      fn,
      fitted,
      residuals,
      metrics: metrics(ys, fitted),
      cond: conditionNumber(A),
    };
  }

  /**
   * Розв'язок через нормальні рівняння (AᵀA)θ = Aᵀb методом Гаусса.
   * Використовується ЛИШЕ для порівняльного аналізу стійкості в тестах/звіті.
   */
  function normalEquationsSolve(xs, ys, m, { normalize = false } = {}) {
    const norm = normalization(xs, normalize);
    const A = designMatrix(xs.map((x) => (x - norm.c) / norm.h), m);
    const n = m + 1;
    const M = Array.from({ length: n }, (_, i) => {
      const row = new Array(n + 1).fill(0);
      for (let j = 0; j < n; j++) for (let r = 0; r < A.length; r++) row[j] += A[r][i] * A[r]
[j];
      for (let r = 0; r < A.length; r++) row[n] += A[r][i] * ys[r];
      return row;
    });
    for (let k = 0; k < n; k++) {
      let p = k;
      for (let i = k + 1; i < n; i++) if (Math.abs(M[i][k]) > Math.abs(M[p][k])) p = i;
```

```
      [M[k], M[p]] = [M[p], M[k]];
      for (let i = k + 1; i < n; i++) {
        const f = M[i][k] / M[k][k];
        for (let j = k; j <= n; j++) M[i][j] -= f * M[k][j];
      }
    }
    const theta = new Array(n).fill(0);
    for (let i = n - 1; i >= 0; i--) {
      let s = M[i][n];
      for (let j = i + 1; j < n; j++) s -= M[i][j] * theta[j];
      theta[i] = s / M[i][i];
    }
    return { theta, fn: (x) => polyEval(theta, (x - norm.c) / norm.h) };
  }

  /**
   * Число обумовленості cond₂(A) = σmax/σmin.
   * σ² — власні значення матриці Грама AᵀA, які знаходимо методом обертань Якобі
   * (для матриць розміру ≤ 21×21 це швидко і достатньо точно для оцінки).
   */
  function conditionNumber(A) {
    const n = A[0].length;
    const M = Array.from({ length: n }, (_, i) => Array.from({ length: n }, (_, j) => {
      let s = 0;
      for (let r = 0; r < A.length; r++) s += A[r][i] * A[r][j];
      return s;
    }));
    for (let sweep = 0; sweep < 100; sweep++) {
      let off = 0;
      for (let p = 0; p < n; p++) for (let q = p + 1; q < n; q++) off += M[p][q] ** 2;
      if (off < 1e-30) break;
      for (let p = 0; p < n; p++) {
        for (let q = p + 1; q < n; q++) {
          if (Math.abs(M[p][q]) < 1e-300) continue;
          const theta = (M[q][q] - M[p][p]) / (2 * M[p][q]);
          const t = Math.sign(theta || 1) / (Math.abs(theta) + Math.sqrt(theta * theta + 1));
          const c = 1 / Math.sqrt(t * t + 1);
          const s = t * c;
          for (let k = 0; k < n; k++) {
            const mkp = M[k][p];
            const mkq = M[k][q];
            M[k][p] = c * mkp - s * mkq;
            M[k][q] = s * mkp + c * mkq;
          }
          for (let k = 0; k < n; k++) {
            const mpk = M[p][k];
            const mqk = M[q][k];
            M[p][k] = c * mpk - s * mqk;
            M[q][k] = s * mpk + c * mqk;
          }
        }
      }
    }
    const eig = M.map((row, i) => Math.max(row[i], 0));
    const lo = Math.min(...eig);
    return lo === 0 ? Infinity : Math.sqrt(Math.max(...eig) / lo);
  }

  /* ---------------------------------------------------------------- метрики */

  function metrics(ys, fitted) {
    const n = ys.length;
    const mean = ys.reduce((a, b) => a + b, 0) / n;
    let sse = 0;
    let sst = 0;
    let maxAbs = 0;
    for (let i = 0; i < n; i++) {
      const r = ys[i] - fitted[i];
      sse += r * r;
      sst += (ys[i] - mean) ** 2;
      maxAbs = Math.max(maxAbs, Math.abs(r));
    }
    return { sse, rmse: Math.sqrt(sse / n), r2: sst === 0 ? 1 : 1 - sse / sst, maxAbs, mean };
  }

  /** Рівномірна сітка з count точок на [a; b]. */
  function linspace(a, b, count) {
    if (count < 2) return [a];
    const step = (b - a) / (count - 1);
    return Array.from({ length: count }, (_, i) => (i === count - 1 ? b : a + i * step));
```

```
  }

  /** Максимальне відхилення двох функцій на сітці (для порівняння з еталоном). */
  function maxDeviation(f, g, a, b, count = 801) {
    let d = 0;
    for (const x of linspace(a, b, count)) d = Math.max(d, Math.abs(f(x) - g(x)));
    return d;
  }

  /** Діапазон значень функції на відрізку (для автоматичного масштабування). */
  function rangeOf(f, a, b, count = 400) {
    let lo = Infinity;
    let hi = -Infinity;
    for (const x of linspace(a, b, count)) {
      const v = f(x);
      if (Number.isFinite(v)) {
        if (v < lo) lo = v;
        if (v > hi) hi = v;
      }
    }
    return { min: lo, max: hi };
  }

  /**
   * Розбір тексту з точками. Підтримувані формати:
   *  • запис варіанта: «(0, 0.0), (1, 0.5), …»;
   *  • по рядку на точку: «x y», «x; y», «x, y», десяткова кома «0,5 0,8».
   */
  function parsePoints(text) {
    const xs = [];
    const ys = [];
    const src = String(text);
    const num = (s, where) => {
      const v = Number(String(s).trim().replace(',', '.'));
      if (s === undefined || String(s).trim() === '' || !Number.isFinite(v)) {
        throw new SyntaxError(`${where}: «${s ?? ''}» не є числом`);
      }
      return v;
    };
    const pairs = [...src.matchAll(/\(\s*([^()]*?)\s*\)/g)];
    if (pairs.length) {
      pairs.forEach((p, i) => {
        let parts = p[1].split(/\s*[,;]\s+|\s*;\s*|\s+/).filter(Boolean);
        if (parts.length === 1) parts = p[1].split(',');
        if (parts.length !== 2) throw new SyntaxError(`Точка ${i + 1}: очікується «(x, y)»,
отримано «${p[0]}»`);
        xs.push(num(parts[0], `Точка ${i + 1}`));
        ys.push(num(parts[1], `Точка ${i + 1}`));
      });
    } else {
      const lines = src.split(/\r?\n/).map((l) => l.trim()).filter((l) => l && !
l.startsWith('#'));
      lines.forEach((line, i) => {
        let parts = line.split(/[;\s]+/).map((p) => p.replace(/,$/, '')).filter(Boolean);
        if (parts.length === 1) parts = line.split(',');
        if (parts.length !== 2) throw new SyntaxError(`Рядок ${i + 1}: очікується «x y»,
отримано «${line}»`);
        xs.push(num(parts[0], `Рядок ${i + 1}`));
        ys.push(num(parts[1], `Рядок ${i + 1}`));
      });
    }
    validatePoints(xs, ys);
    // сортуємо за x, щоб анімація додавання вузлів ішла зліва направо
    const idx = xs.map((_, i) => i).sort((a, b) => xs[a] - xs[b]);
    return { x: idx.map((i) => xs[i]), y: idx.map((i) => ys[i]) };
  }

  /** Форматування полінома за коефіцієнтами при степенях x. */
  function formatPolynomial(coeffs, digits = 4) {
    const terms = [];
    for (let j = coeffs.length - 1; j >= 0; j--) {
      const c = coeffs[j];
      if (Math.abs(c) < 10 ** -(digits + 2)) continue;
      const abs = Math.abs(c).toFixed(digits);
      const pow = j === 0 ? '' : j === 1 ? '·x' : `·x^${j}`;
      terms.push({ sign: c < 0 ? '−' : '+', text: abs + pow });
    }
    if (!terms.length) return '0';
    return terms.map((t, i) => (i === 0 ? (t.sign === '−' ? '−' : '') + t.text : ` ${t.sign} $
{t.text}`)).join('');
```

```
  }

  return {
    validatePoints,
    lagrangeBasis,
    lagrange,
    makeLagrange,
    dividedDifferences,
    newtonEval,
    normalization,
    designMatrix,
    householderLeastSquares,
    polyEval,
    toXCoefficients,
    leastSquares,
    normalEquationsSolve,
    conditionNumber,
    metrics,
    linspace,
    maxDeviation,
    rangeOf,
    parsePoints,
    formatPolynomial,
  };
});
```

Лістинг: js/scale.js

```
/**
 * scale.js — автоматичне масштабування осей.
 *  • computeBounds: обчислює min/max по всіх видимих об'єктах кадру (точки, криві,
 *    залишки) з відступом, щоб графік завжди лишався у видимій зоні;
 *  • niceTicks: «гарні» поділки осі (крок 1·10^k, 2·10^k, 5·10^k);
 *  • approach: плавне наближення поточних меж до цільових (для анімації осей).
 */
(function (root, factory) {
  const api = factory();
  if (typeof module === 'object' && module.exports) module.exports = api;
  else root.Scale = api;
})(typeof self !== 'undefined' ? self : this, function () {
  'use strict';

  /**
   * @param {{xs:number[], ys:number[]}[]} pointSets  набори точок
   * @param {{fn:(x:number)=>number, from:number, to:number}[]} curves  криві з діапазоном x
   * @param {{padX?:number, padY?:number, samples?:number}} opt
   * @returns {{xMin:number, xMax:number, yMin:number, yMax:number}}
   */
  function computeBounds(pointSets, curves, opt = {}) {
    const padX = opt.padX ?? 0.04;
    const padY = opt.padY ?? 0.1;
    const samples = opt.samples ?? 300;
    let xMin = Infinity;
    let xMax = -Infinity;
    let yMin = Infinity;
    let yMax = -Infinity;
    const take = (x, y) => {
      if (!Number.isFinite(x) || !Number.isFinite(y)) return;
      if (x < xMin) xMin = x;
      if (x > xMax) xMax = x;
      if (y < yMin) yMin = y;
      if (y > yMax) yMax = y;
    };
    for (const s of pointSets) s.xs.forEach((x, i) => take(x, s.ys[i]));
    for (const c of curves) {
      if (!(c.to > c.from)) {
        take(c.from, c.fn(c.from));
        continue;
      }
      const step = (c.to - c.from) / samples;
      for (let i = 0; i <= samples; i++) {
        const x = c.from + i * step;
        take(x, c.fn(x));
      }
    }
    if (!Number.isFinite(xMin)) return { xMin: 0, xMax: 1, yMin: 0, yMax: 1 };
    // вироджені діапазони (одна точка / горизонтальна лінія)
    if (xMax - xMin < 1e-12) { xMin -= 0.5; xMax += 0.5; }
```

```
    if (yMax - yMin < 1e-12) {
      const d = Math.max(Math.abs(yMax) * 0.1, 0.5);
      yMin -= d;
      yMax += d;
    }
    const dx = (xMax - xMin) * padX;
    const dy = (yMax - yMin) * padY;
    return { xMin: xMin - dx, xMax: xMax + dx, yMin: yMin - dy, yMax: yMax + dy };
  }

  /** «Гарний» крок сітки для діапазону та бажаної кількості поділок. */
  function niceStep(range, target = 6) {
    const raw = range / Math.max(1, target);
    const pow = 10 ** Math.floor(Math.log10(raw));
    const f = raw / pow;
    const nice = f < 1.5 ? 1 : f < 3 ? 2 : f < 7 ? 5 : 10;
    return nice * pow;
  }

  /** Поділки осі в межах [lo; hi]. */
  function niceTicks(lo, hi, target = 6) {
    if (!(hi > lo)) return { ticks: [lo], step: 1 };
    const step = niceStep(hi - lo, target);
    const start = Math.ceil(lo / step - 1e-9) * step;
    const ticks = [];
    for (let v = start; v <= hi + step * 1e-9; v += step) ticks.push(Math.abs(v) < step * 1e-
9 ? 0 : v);
    return { ticks, step };
  }

  /** Підпис поділки з доречною кількістю знаків після коми. */
  function formatTick(v, step) {
    if (Math.abs(v) >= 1e5 || (Math.abs(v) < 1e-3 && v !== 0)) return v.toExponential(1);
    const digits = Math.max(0, -Math.floor(Math.log10(step) + 1e-9));
    return v.toFixed(Math.min(digits, 6)).replace('-', '−');
  }

  /** Плавне наближення меж: k Є (0;1] — частка шляху за кадр. */
  function approach(cur, target, k) {
    if (!cur) return { ...target };
    const out = {};
    for (const key of ['xMin', 'xMax', 'yMin', 'yMax']) out[key] = cur[key] + (target[key] -
cur[key]) * k;
    return out;
  }

  function boundsClose(a, b, rel = 1e-3) {
    if (!a || !b) return false;
    const sx = Math.abs(b.xMax - b.xMin) * rel;
    const sy = Math.abs(b.yMax - b.yMin) * rel;
    return Math.abs(a.xMin - b.xMin) < sx && Math.abs(a.xMax - b.xMax) < sx
      && Math.abs(a.yMin - b.yMin) < sy && Math.abs(a.yMax - b.yMax) < sy;
  }

  return { computeBounds, niceStep, niceTicks, formatTick, approach, boundsClose };
});
```

Лістинг: js/app.js (фрагмент — побудова сцени кадру)

```
  /**
   * Опис кадру: які криві, точки та залишки видно. Від нього залежить і малювання,
   * і автомасштабування (computeBounds бере саме видимі об'єкти).
   */
  function buildScene() {
    const M = state.model;
    const c = colors();
    const showLag = state.mode !== 'mnk';
    const showMnk = state.mode !== 'lagrange';
    const lo = state.extrap ? M.x0 - 0.25 * M.span : M.x0;
    const hi = state.extrap ? M.xn + 0.25 * M.span : M.xn;
    const scene = { curves: [], points: { added: undefined, active: undefined, pulse: 1 },
residFrac: null, lo, hi };
    const a = state.anim;

    // --- Лагранж
    if (showLag) {
      if (a && a.kind === 'lagrange') {
        const n = M.x.length;
```

```
        const p = Math.min(a.t, n - 1); // p Є [0; n−1]
        const k = Math.min(n - 1, Math.floor(p)); // уже повністю додано k+1 вузлів
        const s = ease(p - k);
        const Pk = M.partial[k];
        if (k + 1 < n) {
          const Pk1 = M.partial[k + 1];
          const right = M.x[k] + s * (M.x[k + 1] - M.x[k]);
          scene.curves.push({ fn: (t) => (1 - s) * Pk(t) + s * Pk1(t), from: M.x0, to: right,
color: c.lag, width: 2.5 });
          scene.points.added = k + 1;
          scene.points.active = k + 1;
          scene.points.pulse = 0.4 + 0.6 * Math.sin(Math.PI * Math.min(1, (p - k) * 1.2));
        } else {
          scene.curves.push({ fn: M.lag, from: lo, to: hi, color: c.lag, width: 2.5 });
        }
      } else if (!(a && a.kind === 'mnk' && a.hideLag)) {
        scene.curves.push({ fn: M.lag, from: lo, to: hi, color: c.lag, width: 2.5 });
      }
    }

    // --- МНК
    if (showMnk) {
      const fitFn = M.fit.fn;
      if (a && a.kind === 'mnk') {
        // t Є [0; 2]: [0;1] — «вирощування» від горизонталі ȳ, [1;2] — поява залишків
        const g = ease(Math.min(1, a.t));
        const mean = M.fit.metrics.mean;
        scene.curves.push({ fn: (t) => (1 - g) * mean + g * fitFn(t), from: lo, to: hi, color:
c.mnk, width: 2.5 });
        const n = M.x.length;
        const r = Math.max(0, a.t - 1) * n;
        scene.residFrac = M.x.map((_, i) => ease(r - i));
      } else if (!(a && a.kind === 'lagrange' && a.hideMnk)) {
        scene.curves.push({ fn: fitFn, from: lo, to: hi, color: c.mnk, width: 2.5 });
        scene.residFrac = M.x.map(() => 1);
      }
    }

    if (state.showRef) {
      scene.curves.unshift({ fn: DATA.reference, from: lo, to: hi, color: c.ref, width: 1.5,
dash: [6, 5] });
    }
    return scene;
  }

  /* ---------------------------------------------------------------------- малювання */

  let rafId = 0;
  let lastTs = 0;

  function requestDraw() {
    if (!rafId) rafId = requestAnimationFrame(frame);
  }

  function frame(ts) {
    rafId = 0;
    const dt = lastTs ? Math.min(0.05, (ts - lastTs) / 1000) : 0.016;
    lastTs = ts;
    let busy = false;

    // крок анімації
    const a = state.anim;
    if (a && !a.paused) {
      a.t += dt * a.rate * state.speed;
      if (a.t >= a.end) {
        a.t = a.end;
        const next = a.queue.shift();
        if (next) startPhase(next, a.queue);
        else finishAnimation();
      }
      busy = true;
    }

    const scene = buildScene();
    const M = state.model;
    const c = colors();

    // автомасштабування: цільові межі по всьому видимому
    const target = S.computeBounds([{ xs: M.x, ys: M.y }], scene.curves, { padX: 0.03, padY:
0.12 });
```

```
    if (state.extrap) {
      target.xMin = Math.min(target.xMin, scene.lo);
      target.xMax = Math.max(target.xMax, scene.hi);
    }
    if (state.snapBounds || !state.bounds) {
      state.bounds = target;
      state.snapBounds = false;
    } else {
      state.bounds = S.approach(state.bounds, target, 1 - Math.exp(-dt * 10));
      if (!S.boundsClose(state.bounds, target)) busy = true;
      else state.bounds = target;
    }
    mainPlot.bounds = state.bounds;
    updateScaleBadge(target);

    mainPlot.clear();
    if (state.extrap) mainPlot.drawExtrapolationZones(M.x0, M.xn);
    mainPlot.drawAxes();
    for (const cv of scene.curves) mainPlot.drawCurve(cv.fn, cv.from, cv.to, cv);
    if (scene.residFrac) mainPlot.drawResidualSegments(M.x, M.y, M.fit.fitted, scene.residFrac,
c.res);
    mainPlot.drawPoints(M.x, M.y, { ...scene.points, highlight: state.hover?.node });

    // підказка / приціл
    const h = state.hover;
    if (h && h.node === undefined && h.x !== undefined) {
      mainPlot.drawCrosshair(h.x);
      for (const cv of scene.curves) {
        if (h.x >= cv.from && h.x <= cv.to && !cv.dash) mainPlot.dot(h.x, cv.fn(h.x), cv.color,
3.5);
      }
    }

    // діаграма залишків (лише для МНК)
    drawResiduals(scene, c);

    if (busy) requestDraw();
    else lastTs = 0;
  }
```

Повний код (index.html, css/style.css, js/chart.js, js/app.js, тести та скрипти) — у репозиторії та архіві проєкту.